Week3

Software architecture

Software architecture refers to the set of principal design decisions, the blueprint of a software system

软件架构指的是一组主要的设计决策，即软件系统的蓝图

Planning the architecture of a software system means understanding how the system should be organized and how to design its overall structure

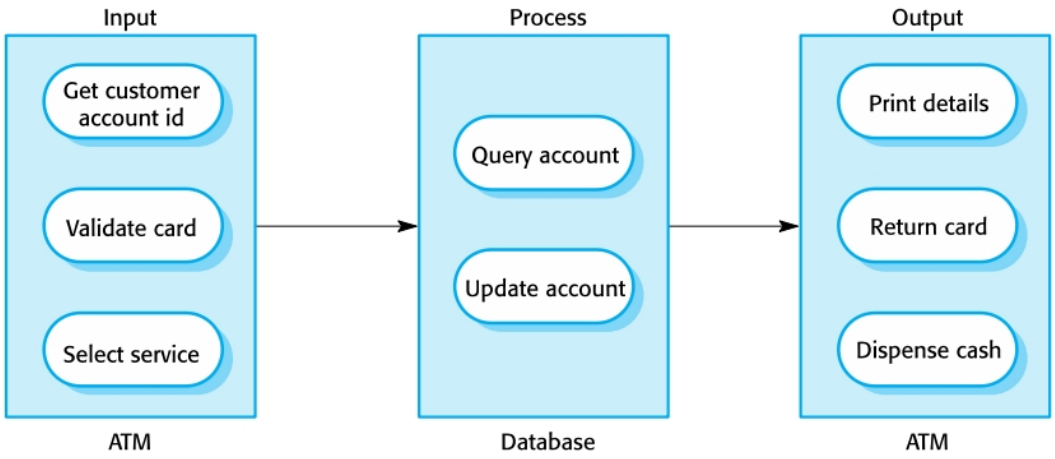
规划软件系统的体系结构意味着理解系统应该如何组织以及如何设计其总体结构

Requirements analysis and software architecture are inevitably connected

需求分析和软件架构是不可避免地联系在一起的

- Non-functional requirements are a result of the architectural choices of the system 非功能性需求是系统架构选择的结果

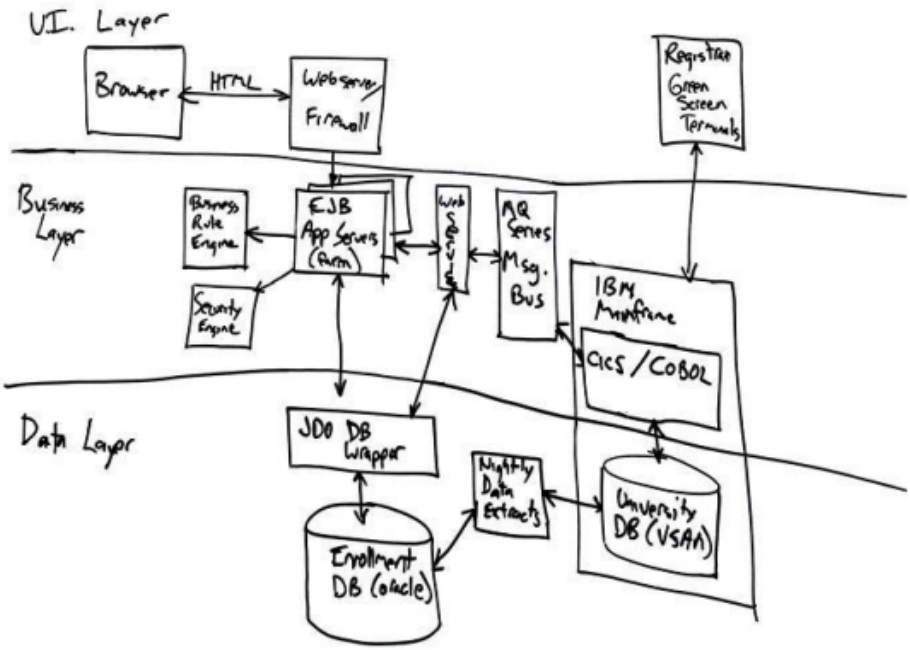
一个ATM的SA的例子



Architectural models 体系结构模型

The architecture is often represented using a simple box and arrow model, or a UML diagram (e.g., component diagram)

架构通常使用简单的方框和箭头模型，或者UML图(例如，组件图)来表示。



Advantages of focusing on architecture

System analysis 系统分析

- The analysis of whether the system can meet its nonfunctional requirements is possible 分析系统是否能够满足其非功能需求是可能的

Large-scale reuse 大规模重用

- The architecture may be reusable across a range of systems (reuse is sometimes an economic necessity) 架构可以跨一系列系统重用(重用有时是经济上的必要条件)。
- Product-line architectures may be developed 可以开发产品线架构

Stakeholder communication 项目干系人的管理与沟通

- Architecture may be used as a focus of discussion by system stakeholders 体系结构可以作为系统涉众讨论的焦点

Agility and architecture 敏捷性和架构

Agile development advocates early delivery and rapid response to change, how can we afford to spend time without coding and designing the architecture instead?

敏捷开发提倡早期交付和对变化的快速响应，我们怎么能在没有编码和设计架构的情况下花费时间呢?

It is generally accepted that an early stage of agile processes is to design the overall architecture of the system

人们普遍认为，敏捷过程的早期阶段是设计系统的整体架构

This is because the system architecture is usually expensive to change because it affects so many important components in the system

这是因为更改系统架构通常代价高昂，因为它会影响系统中的许多重要组件

The architectural design does not need to focus on all design elements of the software but only the most important ones

架构设计不需要关注软件的所有设计元素，只需要关注最重要的设计元素

Architectural patterns (or styles) 架构模式(或风格)

Patterns are a means of representing, sharing and reusing knowledge

模式是表示、共享和重用知识的一种方式

An architectural pattern is a stylized description of good design practice, whichhas been tried and tested in different environments

体系结构模式是对良好设计实践的风格化描述，它已经在不同的环境中进行了尝试和测试

Patterns should include information about when they are and when they are not useful

模式应该包括关于它们何时有用、何时无用的信息

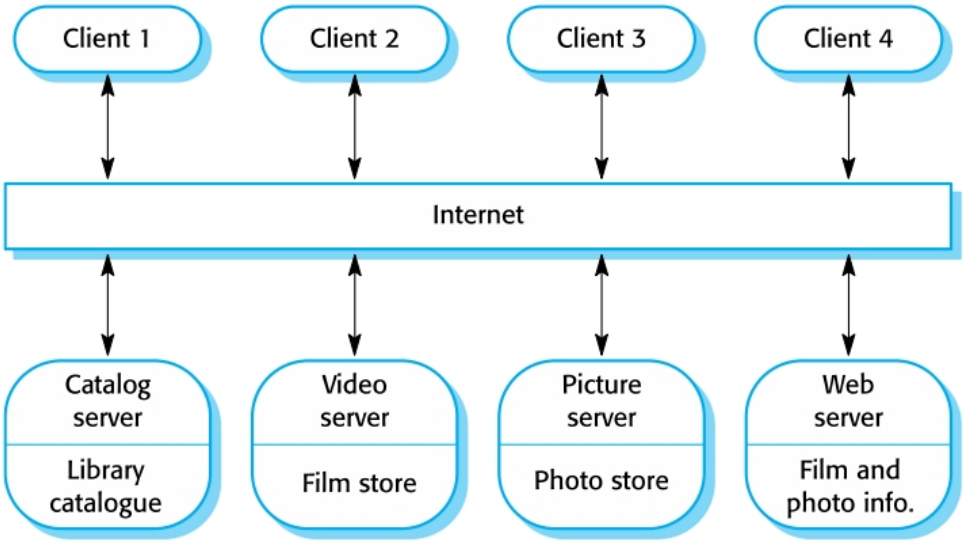
Patterns may be represented using tabular and graphical descriptions

模式可以用表格和图形描述来表示

Web-based architectures

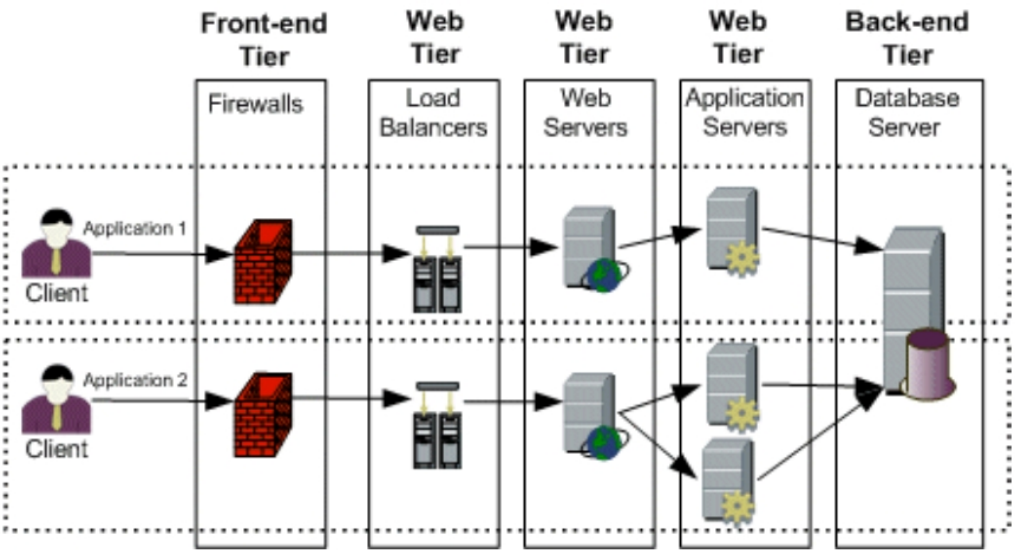
Web-based applications are usually structured according to the **client-server architecture**, potentially organised into **multiple tiers**

基于web的应用程序通常根据客户机-服务器体系结构构建，可能被组织成多个层



（上面是Client，下面是Server）

更具体的看：



Distributed systems architecture 分布式系统架构

Client–server architecture suffers from the presence of a single point of failure and bottleneck

客户机–服务器体系结构存在单点故障和瓶颈

Distributed systems are designed to overcome this by relying on a variety of techniques, including:

分布式系统旨在通过依赖各种技术来克服这一问题，包括:

- Replication and clusters 复制和集群
- Load balancing 负载平衡
- Caching 超高速缓存
- Serverless/cloud computing 无服务架构/云计算
- Hadoop and Map Reduce （两个分布式架构的名字）

Cloud computing

Cloud computing refers to the delivery of on–demand computing services (software and hardware) typically over the Internet

云计算指的是通常通过互联网交付按需计算服务(软件和硬件)

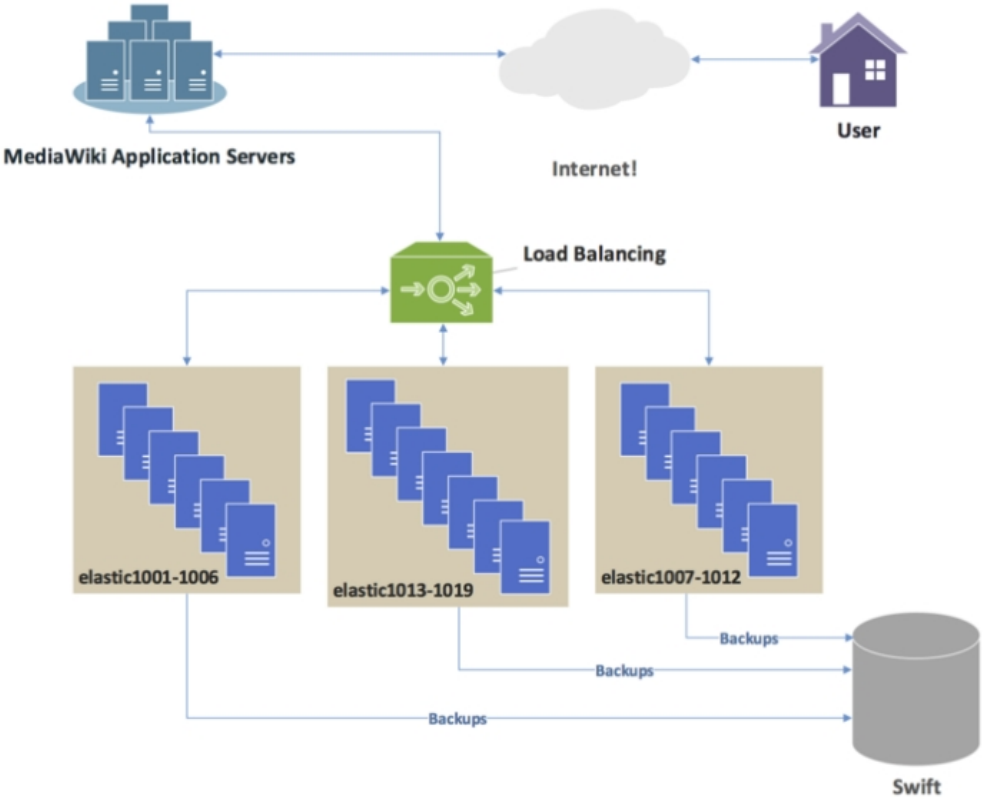
Load balancing

负载均衡，英文名称为Load Balance，其含义就是指将负载（工作任务）进行平衡、分摊到多个操作单元上进行运行，例如FTP服务器、Web服务器、企业核心应用服务器和其它主要任务服务器等，从而协

同完成工作任务。

负载均衡构建在原有网络结构之上，它提供了一种透明且廉价有效的方法扩展服务器和网络设备的带宽、加强网络数据处理能力、增加吞吐量、提高网络的可用性和灵活性。

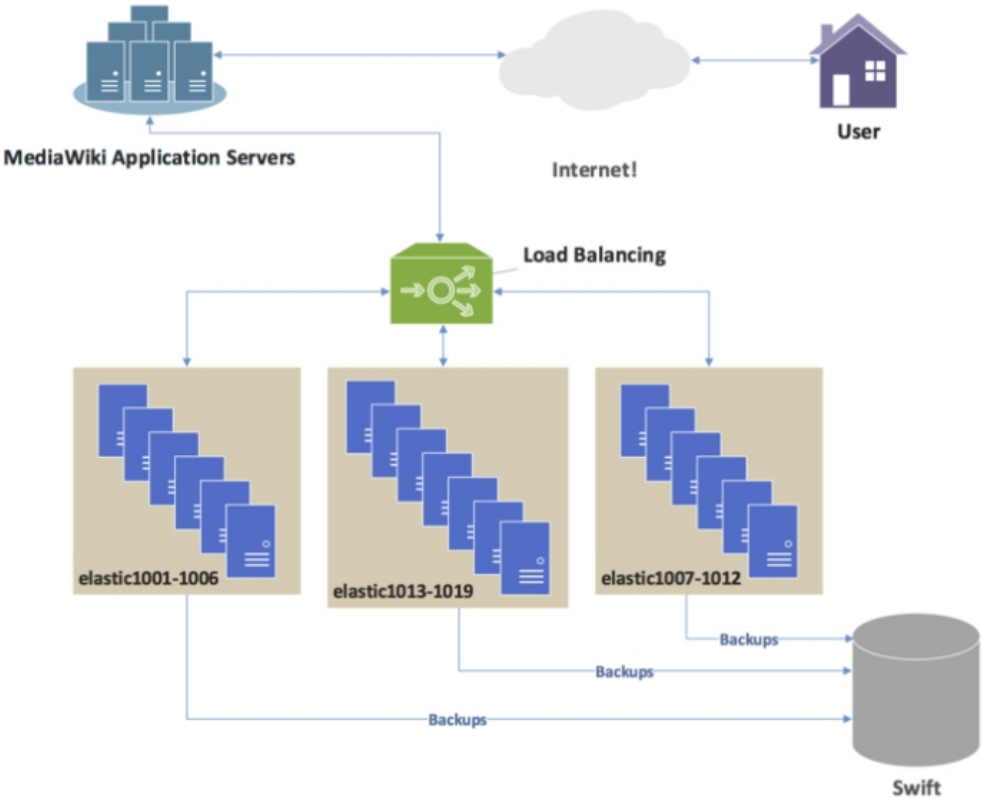
【来源百度百科】



RESTful architecture

Representational state transfer (REST) is a software architectural style that defines how to structure Web services, so that textual representations of Web resources can be accessed and manipulated using a uniform and predefined set of stateless operations

具象状态传输(Representational state transfer, REST)是一种软件体系结构风格，它定义了如何构建Web服务，以便可以使用一组统一的、预定义的无状态操作访问和操作Web资源的文本表示



（利用HTTP Response和Request）

REST is a combination of 6 architectural constraints

REST是6个架构约束的组合

(1) Client–server 主从式架构

(2) Cacheability 可缓存性

(3) Uniform interface 统一接口

(4) Statelessness 无状态性

(5) Layered system 层状体系

(6) Code–On–Demand 可定制代码

Mobile applications architecture

Applications developed for mobiles are usually structured using a layered architecture

为移动设备开发的应用程序通常使用分层架构进行结构化

- Presentation layer 表示层
  - User Interface and UI process components 用户界面和UI流程组件
- Business layer 业务层
  - Main functionalities, potentially deployed on a remote backend to reduce the load on

the mobile 主要功能，可能部署在远程后端，以减少移动设备的负载

- Data layer 数据层
    - Data helpers and utilities, data access components, and service agents 数据帮助程序和实用程序、数据访问组件和服务代理

接下来是Project Management的内容

The need of project management

Software project management is to ensure the software is delivered

软件项目管理就是保证软件的交付

- On time 按时
- Within budget 在预算范围内
- With Quality 有质量

Good management cannot guarantee project success, but bad management usually results in project failure.

好的管理不能保证项目的成功，而不好的管理往往导致项目的失败。

Project manager is responsible for planning and scheduling project, monitoring progress.

项目经理负责计划和安排项目，监督进度。

Software Projects’ Distinctions 软件项目的区别

Software products are intangible and flexible.

软件产品是无形的和灵活的。

Software engineering is not recognised as a sane engineering discipline

软件工程不被认为是一门合理的工程学科

- not standardised 没有标准化的

Many software projects are 'one-off' projects

许多软件项目都是“一次性”项目

- Technologies are changing too fast. 技术变化太快了。
- Previous experiences may be obsolete. 以前的经验可能已经过时了。
- Require a perceptive insight. 需要敏锐的洞察力。

Project Management activities

- Proposal writing 方案制定
- Project planning 项目规划
- Project costing 工程项目成本
- Time management 时间管理
- Project monitoring and reviews 项目监察及回顾
- Personnel selection and evaluation 人员选拔与评价
- Report writing and presentations 报告撰写和演示
- Risk management 风险管理
- Quality management 质量管理

Project planning

To make effective management

进行有效的管理

- Drawn up at the start of the project. 在项目开始时拟定的。
- Probably the most time-consuming activity. 可能是最耗时的活动。

Iterative process

迭代过程

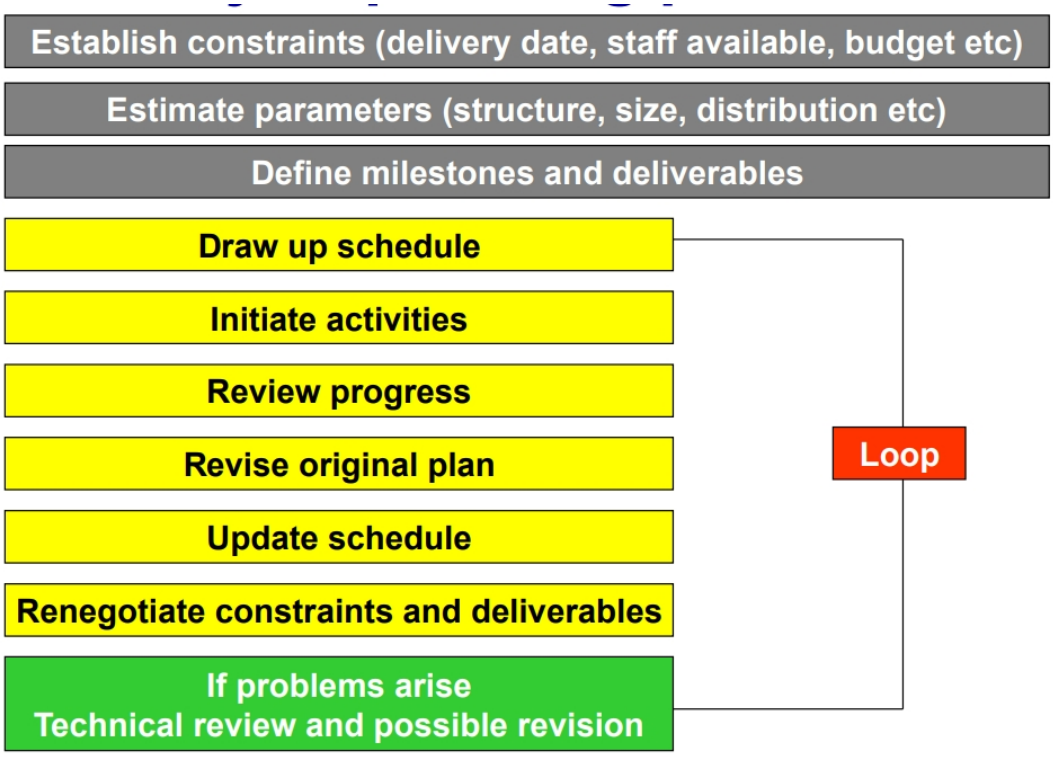
- The plan is only complete when the project itself is complete. 只有当项目本身完成时，计划才算完成。
- Must be regularly revised. 必须定期修改。

Various different types of plan may be developed

可以制定各种不同类型的计划

- Quality plan, staff development plan, … etc 质量计划、员工发展计划等

一个图例：



先“Drawn up at the start of the project”，然后下面的东西就是一次一次的iteration

Types of Project Plan

Quality plan 质量计划

- Describes the quality procedures and standards that will beused in a project. 描述将用于项目的质量程序和标准。

Validation plan 验证计划

- Describes the approach, resources and schedule used for system validation. 描述用于系统验证的方法、资源和时间表。

Configuration management plan 配置管理规划

- Describes the configuration management procedures and structures to be used. 描述要使用的配置管理程序和结构。

Maintenance plan 技术维护计划

- Predicts the maintenance requirements of the system. 预测系统的维护需求。

Staff development plan 职员培训计划

- Describes how the skills and experience of the project team members will be developed. 描述如何发展项目团队成员的技能和经验。

Activity organization

Activities in a project should be organised to. 项目中的活动应组织成

- Produce tangible outputs 产生切实的产出
- Judge progress 判断进展

Milestones are the recognisable end-points of a process activity

里程碑是流程活动的可识别的终点

- Formal output: reports, documentations 正式输出:报告、文档
- Definite: the end of a distinct, logical stage 明确的:一个明确的、合乎逻辑的阶段的结束

Deliverables are project results delivered to customers

可交付成果是交付给客户的项目结果

- Usually delivered at the end of some major phase: specification, design ... etc 通常在一些主要阶段的末尾交付:规范、设计……等

Deliverables are usually milestones, but milestones need not be deliverables!

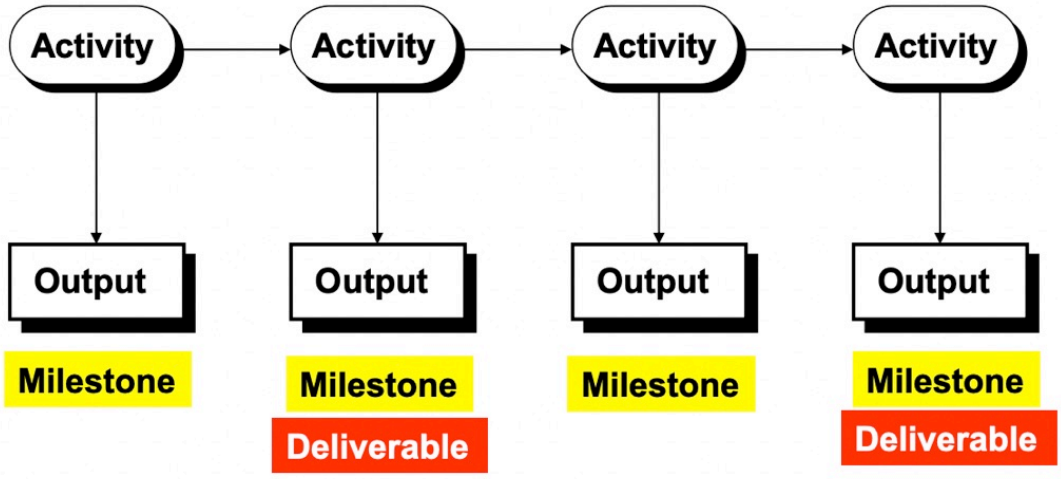
可交付成果通常是里程碑，但里程碑不一定是可交付成果!（不懂的可以看下面子标题的图，会比较容易理解一些）

Establishing milestones

To establish milestones, the software process must be broken down into basic activities with associated outputs.

要建立里程碑，必须将软件过程分解为具有相关输出的基本活动。

- Using a diagram is a good way to represent milestones. 使用图表是表示里程碑的好方法。



-

Project scheduling

Estimate time and resources required to complete activities and organise them into a coherent sequence.

估计完成活动所需的时间和资源，并将它们组织成一个连贯的顺序。

Complicated 复杂的

- Previous estimates are uncertain 先前的估计不确定
- Different design method and implementation language 不同的设计方法和实现语言
- Especially for technically advanced projects 特别是对于技术先进的项目

Project scheduling process

Split project into separate tasks → estimate time and resources required to complete each task.

将项目分成独立的任务→估计完成每个任务所需的时间和资源。

Organise tasks concurrently → to make optimal use of workforce.

同时组织任务→充分利用劳动力。

Minimise task dependencies → to avoid delays caused by one task waiting for another to complete.

最小化任务依赖→避免由于一个任务等待另一个任务完成而造成的延迟。

Dependent on project managers' intuition and experience!

依赖于项目经理的直觉和经验!

Scheduling problems

Do not assume that every stage will be problem free; the unexpected always happens, always allow for a contingency in planning

不要假设每个阶段都没有问题;意想不到的事情总会发生，在计划中一定要考虑到意外情况

- People, hardware, software

A good rule

- Estimate as if nothing will go wrong. 以假如不会出什么差错来估计
- Add 30% for anticipated problems. 为预期的问题增加30%
- Add further 20% to cover unanticipated problems. 再增加20%以覆盖未预料到的问题。

Charts

Project schedules are usually represented as a set of charts

项目进度表通常用一组图表表示

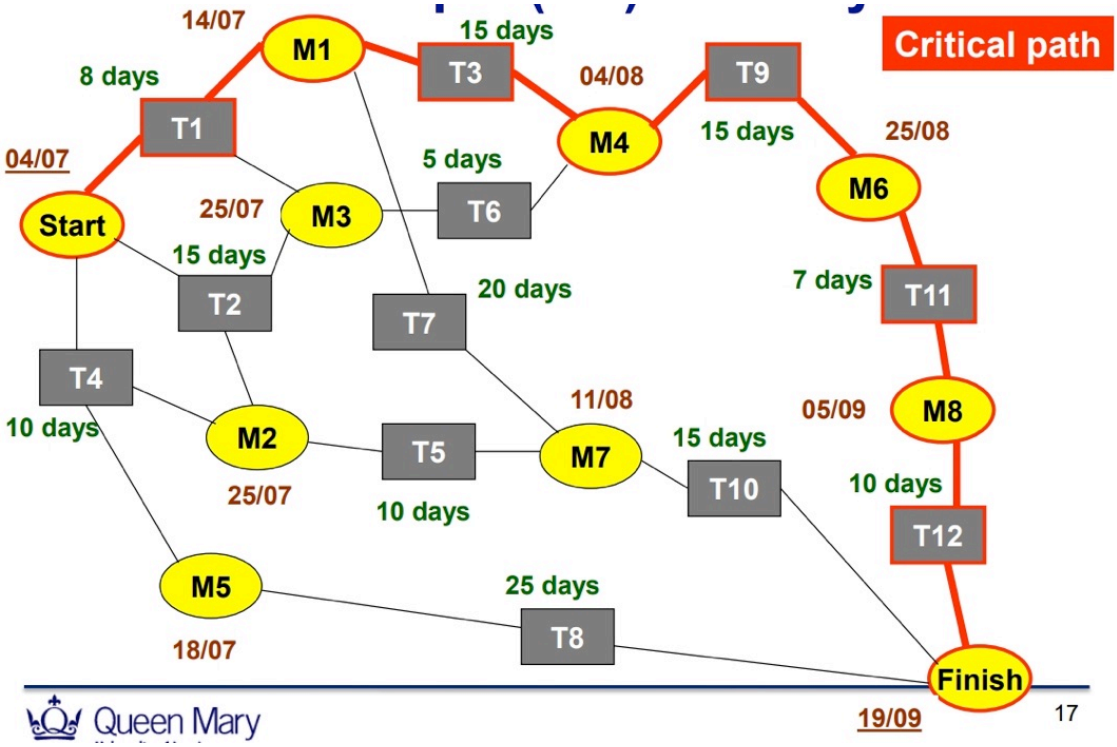
- Work breakdown, activities' dependencies, staff allocations 工作分解，活动的依赖关系，人员分配

Charts are graphical notations used to illustrate the project schedule

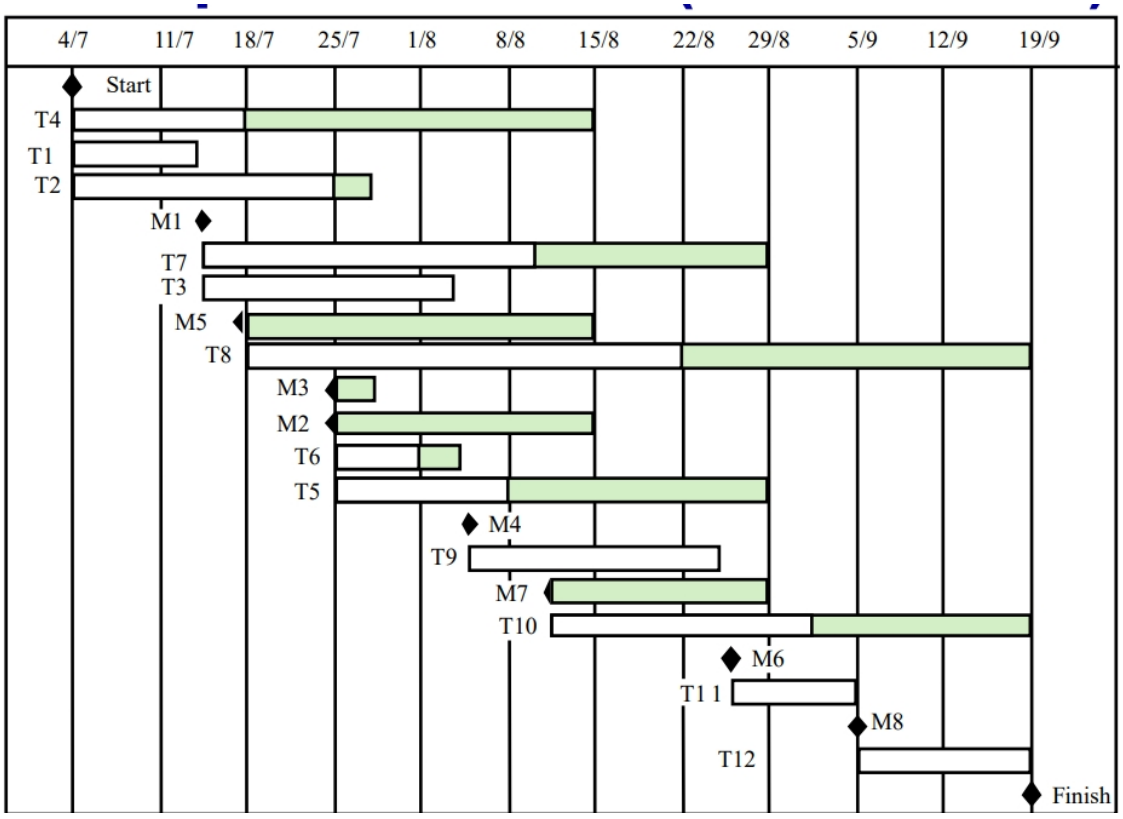
图表是用来说明项目进度表的图形符号

- Task chart: show project breakdown into tasks; tasks should not be too small. 任务图:显示项目分解为任务;任务不应该太小。
Activity	Duration (days)	Dependencies
T1	8	
T2	15	
T3	15	T1 (M1)
T4	10	
T5	10	T2, T4 (M2)
T6	5	T1, T2 (M3)
T7	20	T1 (M1)
T8	25	T4 (M5)
T9	15	T3, T6 (M4)
T10	15	T5, T7 (M7)
T11	7	T9 (M6)
T12	10	T11 (M8)

    T: Task
    M: Milestone
- Activity network: show task dependencies and the critical path. 活动网络:显示任务依赖关系和关键路径。

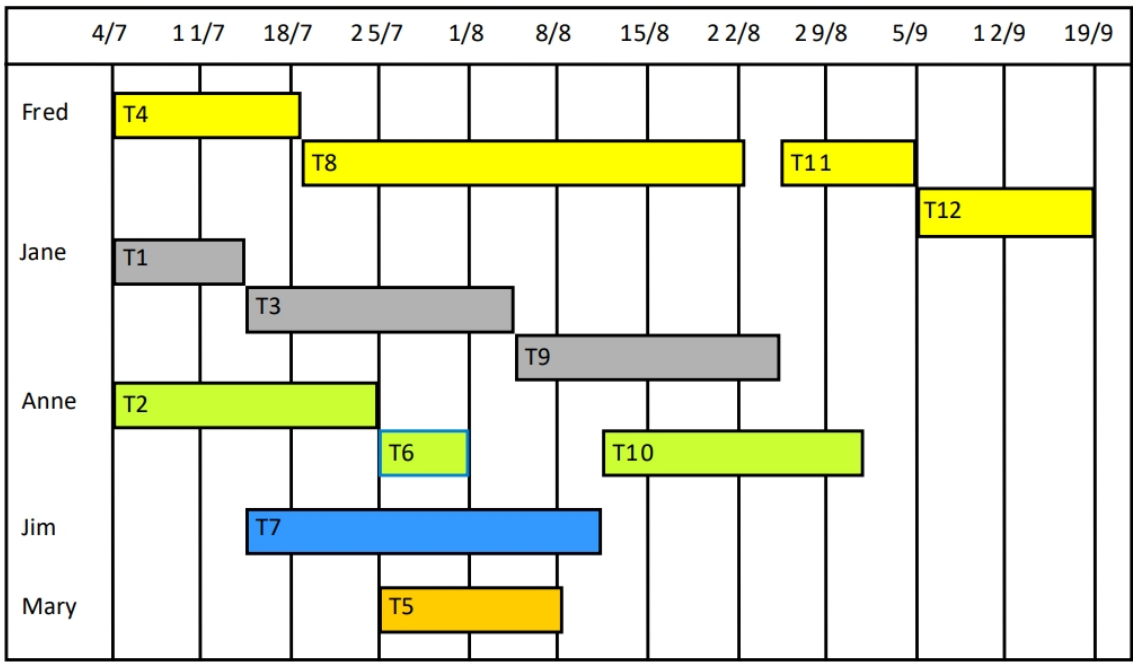


- Bar chart (Gantt Chart): show schedule against calendar time. 条形图:根据日历时间显示时间表。



  - 结合Staff allocation人员编制得出的更详细的bar chart

Written By 顾青弦. 祝你天天开心 学习愉快



Monitoring/Reporting

Project scheduling and project budgeting is often based on prior project experiences and on various product and process measurements (metrics) collected in the past.

项目日程安排和项目预算通常基于先前的项目经验以及过去收集的各种产品和过程度量(度量)。

Daily stand up

每日站会

【 团队每天进行沟通的内部短会，因一般只有15分钟且站立进行而得名。每日站会必须每天在同一时间、同一地点召开，最好的方式是在团队的可视化的任务板前面召开】

【具体看：Daily StandUp(每日站会) – 简书 (jianshu.com)】

Weekly Reports, Weekly Meetings

每周报告，每周会议

Customer Meetings

客户会议

Demos

Metrics 数据记录

Examples of metrics or measurements include:

度量或测量的例子包括:

- Number of lines of code 代码行数

- Number of defects in code (in particular at defined points in development) 代码中的缺陷数量(特别是在开发中的定义点)
- Test cases completed and in what time frame 测试用例已完成，在什么时间范围内完成
- Test cases passed / failed 测试用例通过/失败

Metrics on a 'live' project are also used to assess progress of development against project plans.

"活动"项目的度量标准也用于根据项目计划评估开发进度。

People in the process

People are the most important assets.

人是最重要的资产。

The tasks of a manager:

- Select staff 选择员工
- Motivate people 如何激励别人
- Manage groups 管理组
- Solve technical and non technical problems in the most effective way 以最有效的方式解决技术和非技术问题
- Poor people management is an important contributor to project failure. 糟糕的人员管理是导致项目失败的一个重要因素。

People management factors

Consistency → Team members should all be treated in a comparable way without favourites or discrimination.

一致性→团队成员应该以可比的方式对待，没有偏爱或歧视。

Respect → Different team members have different skills and these differences should be respected.

尊重→不同的团队成员有不同的技能，这些差异应该得到尊重。

Inclusion → Involve all team members and make sure that people's views are considered.

包容让所有团队成员都参与进来，并确保每个人的观点都得到考虑。

Honesty → You should always be honest about what is going well and what is going badly in a project.

诚实→你应该对一个项目中哪些进展顺利，哪些进展不顺利保持诚实。

Managing groups

Most software engineering is a group activity.

大多数软件工程是一种团队活动。

Group interaction is a key determinant of group performance.

群体互动是群体绩效的关键决定因素。

A group is more than a collection of individuals!

一个团体不仅仅是个人的集合!

Group working

Group composition 团体组成

- The right balance of technical skills, experiences and personalities. 技术技能、经验和个性的平衡。

Group cohesiveness 群体凝聚力

- Group spirit: members consider the group to be more important than any individual in it. 团队精神:成员认为团队比任何个人都重要。
- Openness, learn from each other, social events. 开放，取长补短，社会大事。

Group communications 团体沟通

- Size, structure, composition, environment etc. 大小、结构、组成、环境等。

Group organisation 集团组织

- There may be a hierarchical structure for large projects.

Agile project management

The standard approach to project management is plandriven.

项目管理的标准方法是计划驱动的。

Agile project management requires a different approach, which is adapted to incremental development and the particular strengths of agile methods. (Scrum approach)

敏捷项目管理需要一种不同的方法，这种方法适合增量开发和敏捷方法的特殊优势。(Scrum方法)

Teamwork

The ‘Scrum master’ is a facilitator who

“Scrum管理员”是一个促进者

- arranges short daily stand up meetings 安排每日简短的站立会议
- tracks the backlog of work to be done 跟踪待完成工作的积压
- records decisions 记录决策
- measures progress against the backlog 根据待办事项衡量进度
- communicates with customers and management outside of the team. 与客户和团队外的管理层沟通。

Agile team

- In most Agile teams, the golden number is said to be between 5 and 9 members 在大多数敏捷团队中，黄金数字据说在5到9名成员之间

Scrum approach benefits

The product is broken down into a set of manageable and understandable chunks.

产品被分解成一组易于管理和理解的块。

Unstable requirements do not hold up progress.

不稳定的需求不会阻碍进度。

The whole team have visibility of everything and consequently team communication is improved.

整个团队对所有事情都有可视性，因此团队沟通得到了改善。

Customers see on-time delivery of increments and gain feedback on how the product works.

客户可以看到增量的准时交付，并获得关于产品如何工作的反馈。

Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

客户和开发人员之间建立了信任，创造了一种积极的文化，在这种文化中，每个人都期望项目成功。

Risk Management 风险管理

Problems with Projects

Projects do not always follow plans

项目并不总是按照计划进行

Good management involves being prepared for unexpected delays or failures in reaching milestones and producing deliverables

良好的管理包括在达到里程碑和产生可交付成果的过程中为意外的延迟或失败做好准备

We should make alternative plans for:

我们应该制定替代计划:

- Failures which experience suggests often happens 经验告诉我们，失败是经常发生的
- Failures which would have particularly serious consequences 会造成特别严重后果的失败

We cannot plan for everything that can go wrong

我们不能计划所有可能出错的事情

The extent to which we make alternative plans may vary depending on likelihood and consequence

我们制定替代计划的程度取决于可能性和后果

In most cases the alternative plans will have the aim of bringing the project back to its intended goals and timescale

在大多数情况下，备选计划的目的是使项目回到预定的目标和时间范围

It may be necessary to increase the timescale and/or decrease the goals, or even abandon the project

可能需要增加时间范围和/或降低目标，甚至放弃项目

Risk Management是什么

Risk management is the process of identifying possible failures, establishing where alternative plans should be developed, and making those alternative plans

风险管理是识别可能的失败，确定应该在何处制定替代计划，并制定这些替代计划的过程

A risk is a probability of something undesirable happening during development

风险是指在开发过程中发生不希望发生的事情的可能性

Risks can be divided into:

- Project risks : where the schedule or resources available for the project are affected 项目风险:项目的进度或可用资源受到影响
- Product risks : where the quality or performance of the software produced is affected 产品风险:所生产软件的质量或性能受到影响
- Business risks : where the organisation which is developing or procuring the software is affected 业务风险:开发或采购软件的组织受到影响

Some risks may fall into more than one of these categories

有些风险可能不只属于其中一类

Examples of Project Risks

Skilled staff leave, or are absent due to illness

熟练员工请假或因病缺勤

The organisation managing the project changes its priorities: may move staff, reduce available resources, ask for completion in a reduced timetable

管理项目的组织改变其优先级:可能调动人员，减少可用资源，要求在缩短的时间表内完成

Resources expected from other suppliers (hardware, software) are not available on schedule, or do not meet requirements

期望从其他供应商(硬件、软件)获得的资源不能如期获得，或者不能满足需求

Physical environment: building works overrun, transport problems due to poor weather, etc

物理环境:建筑工程超载、恶劣天气造成的交通问题等

Risks which are both Project and Product Risks

Larger number of changes to requirements than anticipated

需求变更的数量比预期的要多

Analysis and specification is not available on schedule

分析和规格不能按期提供

Implementation reveals the size of the system will need to be much larger than anticipated

实施表明，系统的规模需要比预期的大得多

Implementation reveals the development team are lacking in necessary skills and knowledge

实现表明开发团队缺乏必要的技能和知识

Examples of Product Risks

Computer Aided Software Engineering (CASE) tools chosen to support the product do not perform as anticipated

为支持产品而选择的计算机辅助软件工程(CASE)工具没有按预期执行

Serious design error discovered during implementation

在实现过程中发现严重的设计错误

Serious error is not uncovered in testing – logical error, performance, security

在测试中没有发现严重的错误——逻辑错误、性能错误、安全错误

Third party software used in the system performs poorly

系统中使用的第三方软件性能较差

User interface proves unattractive to customers

用户界面对客户没有吸引力

Examples of Business Risks

The technology on which the system is built is superseded by new and better technology

建立系统的技术被新的、更好的技术所取代

New technology means there is less need for the system

新技术意味着对该系统的需求减少了

A similar system developed elsewhere becomes available and is successful in the market before the system is completed

在系统完成之前，在其他地方开发的类似系统已经可用并在市场上取得成功

The organisation responsible for the system experiences financial problems and closes down

负责该系统的组织遇到财务问题并关闭

总结区分三种risk：project，product，business

- project risk指的是推进项目进度的时候遇到的risk，比如员工的问题，管理变动的问题，使员工不能专心做project
- product risk指的是project做出的产品里的问题，比如系统问题，有bug，测试不完善之类的
- business risk指的是卖product的时候的问题，比如财务问题，市场竞争，新技术等等

The Risk Management Process

The tasks involved in risk management are

- Risk Identification 风险识别
    - Identify project, product and business risks 识别项目、产品和业务风险
- Risk Analysis 风险分析
    - Assess the likelihood and consequences of these risks 评估这些风险的可能性和后果
- Risk Planning 风险规划
    - Draw up plans to avoid or minimise the effects of the risk 制定计划以避免或尽量减少风险的影响
- Risk Monitoring 风险监测
    - Monitor the risks throughout the project 在整个项目中监控风险

Avoidance Strategies 规避策略

Be careful about using new or unfamiliar products (tools, software, hardware), balance any

benefit obtained from risk they will not perform as expected

小心使用新的或不熟悉的产品(工具，软件，硬件)，平衡从风险中获得的任何利益，它们将不会像预期的那样运行

But keep awareness of new products, do not stick to old tools and systems out of habit or unwillingness to change

但要保持对新产品的意识，不要出于习惯或不愿意改变而坚持使用旧的工具和系统

Ensure staff are well trained, avoid having just one person who has skills and/or knowledge of a vital topic

确保员工都受过良好的培训，避免只有一个人拥有重要主题的技能和/或知识

Ensure working environment is pleasant, so staff are not tempted to move

确保工作环境令人愉快，这样员工就不会想跳槽

Maintain awareness of the market, new technologies, competing products

保持对市场，新技术，竞争产品的了解

Ensure senior management are well informed on progress of project and its importance to the organisation's goals

确保高级管理层充分了解项目进展及其对组织目标的重要性

Actively monitor performance, be prompt if there is any slippage, do not "hide" problems from senior management or staff

积极监控绩效，如果有任何疏漏，及时告知，不要向高层管理人员或员工"隐瞒"问题

Contingencies 应急计划

Contingencies are alternative plans which are established to be carried out if the original plan cannot be followed

应急计划是指在原计划不能执行的情况下制定的备选计划

They might involve:

- Alternative resources being identified (human, tools, location) 确定替代资源(人、工具、位置)
- Possibility of outsourcing work and resource supply 外包工作和资源供应的可能性
- Priority on which parts of the project to continue, which to postpone or abandon 优先考虑项目的哪些部分要继续，哪些部分要推迟或放弃
- Identifying aspects of the project which could be modified 确定项目中可以修改的方面
- Damage limitation – if a risk that may have already caused problems is discovered what can be done to identify and reduce the effect of those problems? 损害限制–如果发现可能已经引起问题的风险，可以做些什么来识别和减少这些问题的影响?

- Minimisation strategies – what can be done to minimise the future impact of a possible risk? 最小化策略–可以做些什么来最小化可能的风险对未来的影响?

Agile Risk Management

Some aspects of the Agile approach to software development will help with risk management:

敏捷软件开发方法的一些方面将有助于风险管理:

- Agile emphasises continuous testing throughout development, which should enable errors to be detected early 敏捷强调在整个开发过程中持续测试，这应该能够及早发现错误
- Agile emphasises frequent interaction with the customer, so the customer can point out if the system is being developed in a way that seems to be unsatisfactory 敏捷强调与客户的频繁交互，因此客户可以指出系统是否以一种似乎不令人满意的方式开发

Some aspects of the Agile approach, however, may increase risks:

- Agile does not emphasise long–term planning, which may mean it is harder to identify some risks 敏捷不强调长期计划，这可能意味着很难识别一些风险
- Without long–term planning, it is possible that features may be added that cause problems later on 如果没有长期规划，添加的功能可能会在以后引起问题

Quality Management 质量管理

Quality Management是什么

Software quality management is concerned with ensuring a suitable level of quality is achieved in a software product

软件质量管理关注的是确保软件产品达到适当的质量水平

Quality management involves establishing processes and standards within the organisation that will lead to it producing high quality software

质量管理包括在组织内建立过程和标准，从而产生高质量的软件

For individual projects, a separate quality management team should check the deliverables to ensure they are consistent with the organisation's standards and goals

对于单个项目，一个单独的质量管理团队应该检查可交付成果，以确保它们与组织的标准和目标一致

Separating quality management from general development ensures an independent view is taken of the software

将质量管理从一般开发中分离出来，可以确保对软件采取独立的看法

In general, it is hard to be objective when considering the quality of material you have produced: problems are missed due to over–familiarity, poor solutions are kept in if they were

timeconsuming to produce, personal factors amongst team members may impede judgment

一般来说，在考虑你所制作的材料的质量时，很难做到客观:由于过于熟悉而忽略了问题，如果制作费时，则保留了糟糕的解决方案，团队成员之间的个人因素可能会妨碍判断

Agile Quality Management

In the Waterfall approach to software development, quality assurance is seen as a separate process, that takes place after the system has been implemented

在软件开发的瀑布方法中，质量保证被看作是一个独立的过程，发生在系统实现之后

In the Agile approach to software development, with its emphasis on developing software by continuous release of new versions with extra features, quality assurance is seen as a process that takes place at the same time as implementation

在软件开发的敏捷方法中，它强调通过不断发布具有额外功能的新版本来开发软件，质量保证被视为与实现同时发生的过程

Agile development involves a member of the development team having the responsibility of assessing the quality of the system from the point of view of the customer, or actual customer involvement with a customer representative as part of the team

敏捷开发涉及开发团队中的一名成员，该成员负责从客户的角度评估系统的质量，或者实际的客户参与到作为团队一部分的客户代表中

Early release of a system means it can be tested in real use, which often reveals issues that are missed when a system is tested in a way that is made up just for testing

系统的早期发布意味着它可以在实际使用中进行测试，这通常会揭示当系统以一种仅为测试而设计的方式进行测试时遗漏的问题

What is high quality software?

Quality means the software should meet its specifications

质量意味着软件应该满足它的规格

It should be efficient and easy to use

它应该是高效和易于使用的

It should be easy to modify to meet changing requirements

它应该易于修改以满足不断变化的需求

Attention to internal quality of the code – good structure which does not break the main design principles – will greatly contribute to high quality code in terms of meeting customer requirements

关注代码的内部质量——不违背主要设计原则的良好结构——将极大地有助于高质量的代码满足客户的

要求

However, code is “invisible”, customers are unlikely to be directly concerned with “good structure”

然而，代码是“看不见的”，客户不太可能直接关心“良好的结构”。

Customers may find it hard to specify their exact requirements, leading to incomplete and sometimes inconsistent specifications

客户可能会发现很难指定他们确切的需求，从而导致规格不完整，有时甚至不一致

The focus may be **“fitness for purpose”** rather than specification conformance

重点可能是“适合目的”，而不是规范一致性

Fitness for purpose

Have programming and documentation standards been followed in the development process?

在开发过程中是否遵循了编程和文档标准?

Has the software been properly tested?

软件是否经过了适当的测试?

Is the software sufficiently dependable to be put into use?

软件是否足够可靠，可以投入使用?

Is the performance of the software acceptable for normal use?

软件的性能是否可以正常使用?

Is the software usable?

软件可用吗?

Is the software well-structured and understandable?

软件结构良好且易于理解吗?

This can also be expressed as “Is the software **good enough**?”

这也可以表达为“软件足够好吗?”

“Good enough” software

This is an acknowledgement that it is difficult (and expensive) to guarantee software is completely free from errors

这是一种承认，要保证软件完全没有错误是困难的(而且代价高昂)

A balance needs to be achieved between meeting consumer demand for new products or client demand for software to be produced by a deadline, and giving guarantees about performance

需要在满足消费者对新产品的需求或客户对在截止日期前生产软件的需求与提供性能保证之间取得平衡

Software for use in safety critical applications (controlling machinery where an error could result in injury or loss of life) needs higher standards than software for routine administration or entertainment

用于安全关键应用的软件(控制机器，错误可能导致伤害或生命损失)需要比日常管理或娱乐软件更高的标准

Companies producing software need to balance the advantage of bringing new products to market quickly with the risk of damaging reputation if their products are often of poor quality

生产软件的公司需要在将新产品快速推向市场的优势与如果产品质量通常较差而损害声誉的风险之间取得平衡

“Good enough” does not mean lack of attention to good standards during software development: sloppy standards early on will lead to delay and more cost later on as well as poor quality products

“足够好”并不意味着在软件开发过程中缺乏对良好标准的关注:早期草率的标准将导致延迟和后期更多的成本，以及低质量的产品

“Bug or feature?”（辨析概念）

The question “Is it a bug or a feature?” is often asked by users of software systems

“这是一个bug还是一个特性?”这个问题经常被软件系统的用户问到

A “bug” is an aspect of the system’s behaviour which is acknowledged as incorrect

“bug”是系统行为的一个方面，它被认为是不正确的

A “feature” is an aspect of the system’s behaviour which is acknowledged as correct

“特征”是系统行为的一个方面，它被认为是正确的

What about aspects of the system’s behaviour which are neither acknowledged as correct nor as incorrect?

系统行为的某些方面既不被认为是正确的，也不被认为是不正确的呢?

This comes about because it is usually impossible to specify every aspect of behaviour in advance

这是因为通常不可能预先指定行为的每个方面

Unspecified behaviour should be logically consistent with specified behaviour

未指定的行为应该与指定的行为在逻辑上一致

Users may become aware of an inconsistent or unintended behaviour and find it useful in some circumstances, in this way a “bug” becomes a “feature”

用户可能会意识到不一致或意外的行为，并在某些情况下发现它很有用，这样“bug”就变成了“特性”。

It is hard to remove such features, even if they would lead to a better design, if users have come

to rely on them – they may be deprecated in order to give backwards compatibility

如果用户已经开始依赖这些功能，即使它们会带来更好的设计，也很难删除这些功能——为了提供向后兼容性，它们可能会被弃用

Quality Attributes and Trade-offs

The following are all attributes of quality we might want to see in software:

- Safety 安全性
- Understandability 可理解性
- Portability 可移植性
- Security 防范性
- Testability 可测试性
- Usability 可用性
- Reliability 可靠性
- Adaptability 适应性
- Reusability 可重用性
- Resilience 恢复力
- Modularity 模块性
- Efficiency 效率
- Robustness 鲁棒性，健壮性
- Complexity 复杂性
- Learnability 可学习性

【Safety是指由非恶意操作者由于非故意的动作所引起的系统预防的丢失；而Security则是由恶意的操作者由于故意的动作所引起的系统预防的丢失。关键是操作者是否故意。—— 来源：科学网—Security和Safety的区别和统一（140224） – 闵应骅的博文 (sciencenet.cn)】

It is often necessary to trade one off against another, for example extensive run-time checking to ensure resilience and robustness will lead to loss of performance when considering efficiency

通常需要对两者进行权衡，例如，在考虑效率时，为了确保弹性和健壮性而进行的大量运行时检查将导致性能损失

Some of these are “invisible” aspects (such as quality of code), some “visible” (such as quality of user experience)

其中一些是“不可见”的方面(如代码质量)，一些是“可见”的方面(如用户体验的质量)。

Project management should include an assessment of what are the most important quality

attributes for the software being developed, and an agreed way of assessing the extent to which some quality attribute is present in the product

项目管理应该包括对正在开发的软件最重要的质量属性的评估，以及评估产品中某些质量属性存在的程度的商定方法

Visible and Invisible Quality

From the point of view of the customer, the most important quality aspects are what is immediately visible: good user interface and good range of features

从客户的角度来看，最重要的质量方面是直接可见的:良好的用户界面和良好的功能范围

Customer experience of using the system will demonstrate other important quality aspects, such as ease of use of the system, efficiency, quality of its support documentation

客户使用系统的体验将展示其他重要的质量方面，例如系统的易用性、效率、支持文档的质量

Quality aspects such as security and reliability are not immediately visible, but are of direct relevance to the customer

质量方面，如安全性和可靠性，不是立即可见的，但与客户直接相关

Code quality (that is well structured, informed by Clean Code and Design Principles idea) is not of direct relevance to the customer, but is important

代码质量(结构良好，遵循干净的代码和设计原则)与客户没有直接关系，但很重要

Code quality is important because good quality code can be easily modified to add new features, or make required changes

代码质量很重要，因为高质量的代码可以很容易地修改以添加新特性，或者进行必要的更改

Poor quality code is more likely to contain bugs, leading to poor reliability and possible security problems

质量差的代码更有可能包含错误，从而导致可靠性差和可能出现的安全问题

Analysis of Performance

Users generally want software to function “instantaneously”

用户通常希望软件“即时”起作用。

Poor performance is a “bug”, even if the behaviour is logically correct

糟糕的性能是一个“bug”，即使行为在逻辑上是正确的

Like other bugs, poor performance may only occur in unusual circumstances, or in large-scale use, so is missed during testing before release

与其他错误一样，性能差可能只发生在不寻常的情况下，或者在大规模使用中，因此在发布之前的测试中会被忽略

The first defence against poor performance is to employ skilled programmers who are familiar with algorithms

防止性能不佳的第一道防线是雇用熟悉算法的熟练程序员

Also, always use library software for routine algorithms such as sorting

此外，总是使用库软件进行常规算法，如排序

After this, poor performance may come from complex interactions which it is hard to predict in advance

在此之后，较差的性能可能来自于难以提前预测的复杂交互

There are tools which will analyse code execution and show where most of the time is spent

有一些工具可以分析代码执行并显示大部分时间花在哪里

It may not be where you expect

它可能不是你期望的地方

There is little point in improving the efficiency of code which does not contribute much to the execution time, especially if efficiency improvement leads to more complex code

提高代码的效率没有什么意义，因为它对执行时间的贡献不大，特别是当效率的提高导致代码变得更复杂时

Software Reliability

Software reliability is the probability of a software system operating without failure for a specified period of time in a specified environment

软件可靠性是指软件系统在规定的环境下，在规定的时间内无故障运行的概率

A figure for reliability may be part of the requirements specification, but it is difficult to estimate the reliability of a system accurately

可靠性的图表可能是需求规范的一部分，但是很难准确地估计系统的可靠性

Software is not like physical machinery or living things, it does not age or wear out in the sense of it changing over time. However changes in the environment it is used may lead to failure

软件不像物理机器或有生命的东西，它不会随着时间的推移而老化或磨损。然而，使用环境的变化可能导致失败

Software may be unreliable because when it was designed and implemented, care was not taken to ensure it would deal correctly with all situations encountered when used in practice

软件可能是不可靠的，因为当它被设计和实现时，没有注意确保它在实际使用时能够正确处理遇到的所有情况

The problem could be due to the specification not taking account of a situation, or the implementation not meeting the specification

问题可能是由于规范没有考虑到某种情况，或者实现没有满足规范

As with risks in software development, when considering software reliability, two aspects of a software failure need to be considered:

与软件开发中的风险一样，在考虑软件可靠性时，需要考虑软件故障的两个方面:

- Possibility of the failure happening 发生故障的可能性
- Damage caused if the failure happens 故障发生时造成的损坏

Security and Resilience

A major aspect of programming is ensuring that software or the data it uses cannot be damaged by people using it unintentionally in an incorrect way, or people with an intent to damage the interests of legitimate users

编程的一个主要方面是确保软件或它使用的数据不会被无意中以不正确的方式使用它的人或有意损害合法用户利益的人破坏

Software should execute predictably and correctly even when run by someone who wishes to cause damage or access data or services to which they are not entitled

即使有人想要造成破坏或访问他们无权访问的数据或服务，软件也应该可预测地正确执行

Software should contains few or no weaknesses that can be exploited to cause it to stop executing predictably or correctly

软件应该包含很少或没有可以被利用来导致它停止可预测或正确执行的弱点

Software should be able to continue operating even if attempts are being made to use it incorrectly, and it should be able to recover as quickly as possible from attacks made on it

软件应该能够继续运行，即使有人试图错误地使用它，它应该能够尽快从攻击中恢复

Different customers will have different requirements on these issues, security and resilience are less important with software intended for leisure use, but critical for software dealing with financial transactions

不同的客户在这些问题上有不同的需求，安全性和弹性对于用于休闲用途的软件不太重要，但对于处理金融交易的软件至关重要

Software Standards

Standards are a documented set of rules which it is agreed all will keep to

标准是一套成文的规则，大家都同意遵守这些规则

Standards can be international (e.g. ISO 9001), national, organisational or established just for one project

标准可以是国际标准(如ISO 9001)，国家标准，组织标准或仅为一个项目制定的标准

Standards can apply to all aspects of software development, from human management to code layout to documentation

标准可以应用于软件开发的所有方面，从人力管理到代码布局再到文档

Standards aid co-operation – we can work more easily with others if we know the standards they are keeping to

标准有助于合作——如果我们知道别人遵守的标准，我们就能更容易地与他们合作

Standards save time – we can agree to use a well-established set of rules for quality rather than develop our own for each project

标准可以节省时间——我们可以同意使用一套完善的质量规则，而不是为每个项目制定自己的规则

Standards are developed from experience – they incorporate practices which have been seen to work well in the past, and practices which will avoid mistakes that were made in the past

标准是从经验中发展出来的——它们结合了过去行之有效的做法，以及可以避免过去所犯错误的做法

Standards help new staff understand the organisation and provide a framework of good practice for them to work in

标准帮助新员工了解组织，并为他们提供一个良好实践的框架

Problems with Standards

Standards can become out of date due to technology changes or changes in experience

由于技术的变化或经验的变化，标准可能会过时

For example, standards may call for action against problems which no longer occur as they were caused by technology which is no longer used

例如，标准可能要求对不再出现的问题采取行动，因为这些问题是由不再使用的技术引起的

Experience may show that a safeguard against one problem causes more problems elsewhere

经验可能表明，针对一个问题的防范措施会在其他地方引发更多问题

Standards may be seen as bureaucratic form-filling which wastes time

标准可能被视为浪费时间的官僚填表

Good aspects of standards may not be taken seriously if there are also poor aspects

如果标准也有不好的方面，那么好的方面可能不会被认真对待

So standards should be reviewed regularly, with current practitioners involved in the review and the rationale clearly explained

因此，应该定期对标准进行审查，让当前的从业人员参与审查，并清楚地解释其基本原理

Standards should be supported by tools to minimise any extra clerical work, but poor tools can lead to even more overhead

标准应该得到工具的支持，以尽量减少任何额外的文书工作，但糟糕的工具可能导致更多的开销

Software Engineering History部分省略，有兴趣请自行去Topic14 28页开始学习，此专题下只摘取一点我觉得有用的东西

Code Smells and Design Patterns

The term code smell to mean an indication in code of a need for refactoring

术语“代码气味”是指代码中需要重构的指示

A “code smell” is not necessarily an indication of poor design or coding in the first place, it is a recognition that aspects of design which made sense at one stage can become poor as the code is modified to accommodate new features

“代码气味”一开始并不一定是糟糕的设计或编码的指示，它是一种认识，即在一个阶段有意义的设计方面可能会随着代码修改以适应新功能而变得糟糕

Agile development recommends keeping code simple, so do not introduce complexities into design if they are not immediately needed

敏捷开发建议保持代码简单，所以如果不是立即需要，不要在设计中引入复杂性

As the code becomes more complex, we may see ways in which we can break the code into parts and generalise it in order to keep to the design principles

随着代码变得越来越复杂，我们可能会看到将代码分成几个部分并将其一般化以保持设计原则的方法

Design patterns are particular ways of structuring code to do this

设计模式是构建代码以实现此目的的特定方法

However, excessive use of design patterns before the growth in requirements make it useful can make code too complex, which is itself a “smell”

然而，在需求增长之前过度使用设计模式会使代码过于复杂，这本身就是一种“气味”。

Code Smell Examples

Long methods: if the code for a method has become so long that it cannot be completely viewed on a screen, that is a sign the method could be broken into several methods

长方法:如果一个方法的代码太长，以至于无法在屏幕上完整显示，这表明该方法可能被分解成几个方法

Large classes: if a class has many methods and fields that is a sign it could be broken into separate classes

大类:如果一个类有很多方法和字段，这是一个信号，它可以被分解成单独的类

Data clumps: if many methods take the same values as several arguments, that is a sign that putting those values together as fields of an object and passing the object as a single argument

would be a good idea

数据团块:如果许多方法将相同的值作为几个参数，这表明将这些值放在一起作为对象的字段并将对象作为单个参数传递将是一个好主意

Refuse Bequest: if a class extends another class, but does not make use of what it inherits, or overrides its methods with code that serves no purpose, that is a sign that inheritance was not a good technique to use for that class

拒绝继承:如果一个类继承了另一个类，但是没有使用它继承的东西，或者用没有目的的代码重写了它的方法，这就表明继承不是一个很好的技术，不适合这个类

Duplicate Code: if a class has a lot of code that is similar to that of another class, that is a sign that techniques should be used to avoid duplicated code, in that case it might be appropriate to use inheritance so the two classes inherit code from a superclass

重复代码:如果一个类有很多与另一个类相似的代码，这是一个应该使用技术来避免重复代码的信号，在这种情况下，使用继承可能是合适的，所以两个类从超类继承代码

到现在了，我们终于可以看看本门课程的标题“software engineering”到底是个什么玩意了

What is Software Engineering?

All aspects of software development:

软件开发的各个方面:

- Requirements analysis 需求分析
- Design 设计
- Implementation 执行
- Testing 测试

But also

- Management of human resources 人才资源管理
- Business issues 商业问题
- Use of CASE tools (Computer Aided Software Engineering) 使用CASE工具(计算机辅助软件工程)
- Modification of existing software 修改现有软件
- Ethical issues 伦理问题

就是这些玩意加起来，不止有coding的问题

Trade offs 权衡

Software MUST

- Meet customer requirements 满足顾客的要求
- Be convenient to use 好使
- Be safe 安全
- Be efficient 高效
- Be secure 高防范性
- Meet legal requirements 符合法律要求

But also

- Has to be produced within a time limit 必须在限定时间内生产
- Has to be produced using available human resources 必须利用现有的人力资源来生产
- Has to be produced using available software resources 必须使用可用的软件资源进行生产
- Has to be produced using available financial resources 必须使用现有的财政资源来制作

Software lifespan 软件生命周期

Most software systems are under continuous development:

大多数软件系统都在持续开发中:

- Customer requirements change 客户需求变更
- Experience with use suggests new features 使用经验暗示了新特性
- Errors or poor design features need to be corrected 错误或糟糕的设计特性需要纠正
- Advances in technology open up new possibilities 技术的进步带来了新的可能性

Software scale

Most real world software systems are large scale:

大多数现实世界的软件系统都是大规模的:

- Millions of lines of code
- Divided into many classes
- Produced by many people
- Expanded by many revisions

Design principles 设计原则

Attention to high quality design and implementation is essential if we are to manage the complexity of modern real world software systems

如果我们要管理现代现实世界软件系统的复杂性，那么关注高质量的设计和实现是必不可少的

We cannot approach software development in the casual way we may have used when we first learnt to program

我们不能像第一次学习编程时那样随意地进行软件开发

We have to think of software systems at a higher level than just code

我们必须在比代码更高的层次上考虑软件系统

When we write code, it is not enough that it works correctly, it must also be written in a way that makes it easy to develop further

当我们编写代码时，仅仅正确地工作是不够的，它还必须以一种易于进一步开发的方式编写

The dangers of poor quality code

We cannot be sure it works correctly

我们不能确定它是否能正常工作

If we change one part of it to meet a new requirement, we cannot be sure whether that will cause other parts to stop working correctly

如果我们改变它的一部分来满足一个新的要求，我们不能确定这是否会导致其他部分停止正常工作

When we change one part to meet a new requirement, we find we have to make many changes elsewhere

当我们改变一个部分来满足一个新的要求时，我们发现我们必须在其他地方做很多改变

We have to understand every part of the code to understand how any part of it works

我们必须理解代码的每一部分，才能理解其中任何一部分是如何工作的

Every change made causes all these problems to grow worse

每一次改变都会导致所有这些问题变得更糟

As modern software is long-lived and undergoes many revisions, a poor design decision made early on can have very expensive consequences later

由于现代软件的寿命很长，并且要经历多次修改，因此早期做出的糟糕设计决策可能会在以后造成非常昂贵的后果

Security problems with code

Programs in most cases are not self-contained, they interact with other programs

程序在大多数情况下不是自包含的，它们与其他程序相互作用

Interaction can lead to security problems, where one program can gain access to data in another program that is meant to be secure, and make changes to it that should not be made

交互可能导致安全性问题，其中一个程序可以访问另一个程序中的数据，而这些数据本应是安全的，并对其进行不应该进行的更改

A security problem may be unintentional, meaning it was not realised by the programmers that one piece of code would interact with another to make an unexpected change

安全问题可能是无意的，这意味着程序员没有意识到一段代码会与另一段代码交互，从而产生意想不到的更改

Security problems may be intentional: deliberate use of code to gain unauthorised access

安全问题可能是故意的:故意使用代码来获得未经授权的访问

Simple examples of where code can lead to security problems are given later:

后面给出了代码可能导致安全问题的简单示例:

- Exposing the representation 公开表示
- Liskov Substitution Principle Liskov代换原则/里氏代换原理

Decomposition 分解

To manage large scale software we have to divide a system into separate parts (“modules”), in a way that means as far as possible each part can be considered on its own. The aim is to give:

为了管理大规模的软件，我们必须将系统划分为独立的部分(“模块”)，在某种程度上意味着每个部分都可以单独考虑。其目的是:

- Locality – the implementation of a module can be read or written without having to examine the implementation of any other module 局部性——一个模块的实现可以被读取或写入，而不必检查任何其他模块的实现
- Modifiability – a module can be reimplemented without requiring changes to any module which makes use of it 可修改性——一个模块可以被重新实现，而不需要改变使用它的任何模块

Specification and implementation

We design a software system in terms of modules which are defined by the services they provide to other modules

我们根据模块来设计软件系统，这些模块由它们为其他模块提供的服务来定义

We implement a module so that it provides the services it is required to provide

我们实现一个模块，以便它提供它需要提供的服务

Specification is the link between these

规范是两者之间的纽带

Implementing a module is done in the same way: consider how it could be split into separate parts (“top down” approach)

实现模块的方式也是一样的:考虑如何将其分成不同的部分(“自顶向下”方法)。

To maintain locality and modifiability it is important that modules cannot interact in any way that is outside their specification

为了维护局部性和可修改性，重要的是模块不能以其规范之外的任何方式进行交互

接下来更多是一些JAVA的基础知识与软工结合

Classes and Methods

You could think of “module” here as meaning class, as the standard way of dividing code into parts in objectoriented programming is to divide it into separate classes

您可以将这里的“模块”理解为类，因为在面向对象编程中，将代码分成部分的标准方法是将其分成单独的类

The code for a class is itself divided into code for methods

类的代码本身分为方法的代码

The way in which code interacts is for code in a method in one class to call a method from another class

代码交互的方式是让一个类中方法中的代码调用另一个类中的方法

A method in one class can also call a method from the same class

一个类中的方法也可以调用同一类中的方法

Object-oriented programming is based on the concept of calling a method on an object, with the code for the method being in the class of the object.

面向对象编程基于在对象上调用方法的概念，方法的代码位于对象的类中。

Separation

Treat independently WHAT an entity does from HOW it does it

独立对待一个实体做了什么和它是如何做的

In object oriented programming, WHAT an object does is given by the specification of its methods

在面向对象编程中，对象的功能是由其方法的说明给出的

HOW an object does it is given by the code for its methods and the variables that form its internal structure

一个对象是如何由它的方法和构成它内部结构的变量的代码给出的

Separation allows us to design with objects at application level (using them) and implementation level (what makes them work)

分离允许我们在应用程序级别(使用它们)和实现级别(使它们工作的内容)对对象进行设计。

It is much easier if we can think of one without having to think of the other at the same time

如果我们能想到其中一个而不必同时想到另一个，那就容易多了

Splitting Methods and Classes 拆分方法和类

Planning separate classes and methods before implementation is one aspect of keeping code well structured

在实现之前规划单独的类和方法是保持代码良好结构的一个方面

Splitting classes and methods as code becomes complex is another aspect of keeping code well structured

当代码变得复杂时拆分类和方法是保持代码良好结构的另一个方面

This is particularly important with the Agile approach, which involves frequent modification of code to meet new requirements

这在敏捷方法中尤为重要，因为敏捷方法需要经常修改代码以满足新的需求

If the code for a method is long and complex, consider whether some of it can be put into separate methods which perform subtasks

如果一个方法的代码又长又复杂，请考虑是否可以将其中的一些代码放入执行子任务的单独方法中

If several variables used are related, consider whether it makes sense to combine them in an object defined by a separate class

如果使用的几个变量是相关的，请考虑将它们组合在由单独类定义的对象中是否有意义

Helper Methods and Classes

Declaring variables in a class as private is needed so that code in other classes cannot access them directly, to keep to the principle that only specified interaction can occur

需要将类中的变量声明为私有，这样其他类中的代码就不能直接访问它们，以遵守只有指定的交互才能发生的原则

If you have defined a separate method just to help split your code into parts, it is called a helper method

如果您定义了一个单独的方法，只是为了帮助将代码分成几个部分，那么它被称为辅助方法

A helper method should be declared as private, because code in other classes should not be able to call it directly, as that would also be outside the specification

助手方法应该声明为私有，因为其他类中的代码不应该能够直接调用它，因为这也超出了规范

Java allows a class to be declared inside another class (a nested class), and declared as private so it can only be used directly by code in the class it is in

Java允许在另一个类(嵌套类)中声明一个类，并将其声明为私有，因此只能由它所在类中的代码直接使用

Reuse

Instead of writing new code for a module to provide some required service, we may make use of existing code. This has the advantage of:

我们可以利用现有的代码，而不是为模块编写新的代码来提供一些所需的服务。这样做的好处是:

- Saving the effort of writing new code 节省了编写新代码的工作量
- The existing code may already be well developed and tested for errors 现有的代码可能已经很好地开发和测试了错误
- The existing code may be a well-understood standard 现有的代码可能是一个很容易理解的标准
- Overall code size is smaller if code is reused in several places (this is not “cut and paste” reuse!) 如果代码在多个地方被重用，总代码大小会更小(这不是“剪切和粘贴”重用!)
- When re-used code is updated, all the code that uses it shares in the update 当重用的代码更新时，所有使用它的代码共享更新

Code for Reuse

Code for reuse may be:

- Provided as standard with our programming language 作为我们编程语言的标准提供
- Provided by a “third party” supplier 由“第三方”供应商提供
- Developed as an “in house” code library 作为“内部”代码库开发
- Adapted from other parts of the project 改编自项目的其他部分
- Developed intentionally early on in the project as it is anticipated such a service will be required (“bottom up”approach) 在项目早期有意开发，因为预计会需要这样的服务(“自下而上”方法)。

Abstraction and generalisation

Abstraction is identifying the essential aspects of some situation in order to model it in software

抽象是识别某些情况的基本方面，以便在软件中对其建模

Generalisation is identifying common elements in software requirements so we can provide them in reusable code

泛化是识别软件需求中的公共元素，以便我们可以在可重用的代码中提供它们

Ability and confidence with abstraction and generalisation are key skills for expert programmers

抽象和泛化的能力和信心是专家级程序员的关键技能

The growing emphasis on reusability in code makes abstraction and generalisation particularly important

对代码可重用性的日益强调使得抽象和泛化变得尤为重要

Code written in an abstract style can be hard to follow, do not over-use to “show off”

用抽象风格编写的代码可能很难遵循，不要过度使用“炫耀”。

UML diagrams v. Java code

It is important to see software at a higher level than code, this is a form of abstraction

在比代码更高的层次上看待软件是很重要的，这是一种抽象形式

UML diagrams are a standard way of expressing the design of software at a higher level than code

UML图是在比代码更高的层次上表达软件设计的标准方法

Object oriented programming languages have become the standard for general purpose programming

面向对象编程语言已经成为通用编程的标准

Object oriented programming language code relates closely to UML class diagrams

面向对象编程语言代码与UML类图密切相关

Java is currently the most popular object oriented programming language, C++ and C# are similar to it

Java是目前最流行的面向对象编程语言，c++和c#都与之相似

UML class diagrams

UML class diagrams correspond particularly closely to Java code (and to code in similar languages)

UML类图与Java代码(以及用类似语言编写的代码)特别紧密地对应。

The nodes in UML class diagrams correspond to Java classes and interface types

UML类图中的节点对应于Java类和接口类型

The nodes are annotated with information corresponding to the fields and methods of a class

节点用与类的字段和方法相对应的信息进行注释

Arcs represent relationships between classes, IS–A relationships are inheritance, HAS–A relationships are where a class has a field of a particular type

弧表示类之间的关系，IS–A关系是继承，HAS–A关系是类具有特定类型字段的地方

UML enables documentation on intended HAS–A relationships between classes beyond what can be expressed in Java

UML支持超出Java所能表达的类之间预期的HAS–A关系的文档

Static v. Dynamic

Static means those aspects of code that are to do with its permanent structure

静态意味着代码的那些方面与其永久结构有关

Dynamic means those aspects of code that are to do with when the code is run

动态是指代码运行时与之相关的那些方面

Class diagrams in UML match the static view of code, sequence diagrams match the dynamic view

UML中的类图匹配代码的静态视图，序列图匹配动态视图

In an object oriented programming language, a class is a static aspect, an object is a dynamic aspect

在面向对象的编程语言中，类是静态方面，对象是动态方面

Another use of the word “static” is seen in the Java keyword static

“static”一词的另一种用法见于Java关键字static

Classes v. Objects

A class may be considered a “recipe” or “blueprint” or “description” of a type of object

一个类可以被看作是一个对象类型的“配方”、“蓝图”或“描述”

Any number of objects may be produced to the description given in a class

任何数量的对象都可以根据类中给出的描述产生

We describe an object as an instance of its class

我们将对象描述为其类的实例

Each object of a particular class has its own variables of the names and types declared in the class

特定类的每个对象都有自己在类中声明的名称和类型的变量

A similar relationship can be considered between a method(static concept) and a method call (dynamic concept)

方法(静态概念)和方法调用(动态概念)之间也可以考虑类似的关系。

Objects and method calls

An object contains data in its own instance variables

对象在它自己的实例变量中包含数据

Method calls have their own parameter and local variables

方法调用有自己的参数和局部变量

So a method call on an object has access to the instance variables of that object and parameter and local variables created just for that method call

一个对象上的方法调用可以访问那个对象的实例变量以及为那个方法调用创建的参数和局部变量

Method calls may:

- Return values (primitive or objects) or throw exceptions 返回值(原语或对象)或抛出异常
- Change the values of the instance variables of the object they are called on 更改调用对象的实例变量的值
- Have an external effect (input/output) 具有外部效果(输入/输出)
- Call methods on objects referred to by the instance variables 调用实例变量所引用的对象的方法
- Call methods on this 调用方法
- Call methods on objects referred to by parameter variables 对参数变量引用的对象调用方法
- Call methods on objects constructed by the method call 在由方法调用构造的对象上调用方法

Java's keyword static

If a method is declared as static in Java, it means the method is called on its own rather than called on an object

如果一个方法在Java中被声明为静态，这意味着该方法是单独调用的，而不是在对象上调用的

A static method from another class can be called attached to its class, so long as it is not declared as a private method

来自另一个类的静态方法可以被附加到它的类中调用，只要它没有被声明为私有方法

If a variable is declared as static, it means instead of every object having its own variable of that name, there is just one variable of that name shared by all objects of its class

如果一个变量被声明为静态，这意味着不是每个对象都有自己的同名变量，而是只有一个同名变量被其类的所有对象共享

If one object changes the value of a static variable then another object makes use of its value, that is another form of object interaction

如果一个对象改变了静态变量的值，那么另一个对象就会使用它的值，这是另一种形式的对象交互

Interaction by using shared variables makes code hard to follow, so is generally not a good idea

通过使用共享变量进行交互会使代码难以理解，因此通常不是一个好主意

Variables and referencing

Variables in Java refer to objects

Java中的变量引用对象

A method call attached to a variable in the code means it is called on the object that the variable refers to at that point

代码中附加到变量的方法调用意味着在该变量引用的对象上调用该方法

More than one variable may refer to an object, this is called aliasing

多个变量可以引用一个对象，这称为混叠

A variable may change which object it refers to, so if var1 and var2 are variables, the assignment var2=var1 means “var2 stops referring to what it used to refer to and starts referring to what var1 refers to”.

变量可以改变它所引用的对象，因此如果var1和var2是变量，则赋值var2=var1意味着“var2停止引用它以前引用的对象，并开始引用var1所引用的对象”。

If var2 and var3 are aliases, var2=var1 does not change what var3 refers to, so it causes var2 to stop being an alias to var3 and start being an alias to var1

如果var2和var3是别名，var2=var1不会改变var3所指的内容，因此它导致var2不再是var3的别名，而开始成为var1的别名

Change through an alias is shared

通过别名进行的更改是共享的

【PPT Topic15 P26有图，P33有例子，不理解可以看】

【查了一下，alias翻译成“混叠”“别名”的都有，这里就懒得改了，知道是alias就行】

Scope and garbage collection 作用域和垃圾收集

In Java, an object, with all its instance variables, remains in existence so long as there is at least one variable which refers to it

在Java中，只要至少存在一个引用对象的变量，对象及其所有实例变量就保持存在

A parameter variable remains in existence until the method call which brought it into existence finishes

参数变量一直存在，直到使其存在的方法调用结束

A local variable remains in existence until execution reaches the end of the block of code it is in (the matching } to the first { before it was declared)

局部变量一直存在，直到执行到它所在的代码块的末尾(与声明之前的第一个{匹配})。

If there is no variable referring to an object it is automatically “garbage collected” (in some programming languages the programmer must write code to do this)

如果没有变量引用对象，它将自动“垃圾收集”(在某些编程语言中，程序员必须编写代码来完成此操作)。

An object can be created in a method call but remain in existence after the method call, because it is returned by that method call or an instance variable is set to refer to it

对象可以在方法调用中创建，但在方法调用后仍然存在，因为该对象由该方法调用返回，或者设置了一个实例变量来引用它

Care with Aliasing

Problems can be caused when objects whose state can be changed by method calls are aliased

当可以通过方法调用更改状态的对象被别名时，可能会导致问题

When writing code that makes use of an object, we need to take into account any possibility that some other code that aliases it may also make changes to it

当编写使用对象的代码时，我们需要考虑到别名它的其他代码也可能对它进行更改的可能性

When testing code that has two object parameters of the same type, we need to test that it behaves correctly when they are aliases

当测试具有相同类型的两个对象参数的代码时，我们需要测试当它们是别名时它的行为是否正确

It is good if possible to make objects immutable (no state changes possible) as there are then no aliasing problems

如果可能的话，最好使对象不可变(不可能改变状态)，因为这样就不会出现混叠问题

Take particular care when returning references to internal objects, or incorporating references to external objects

在返回对内部对象的引用或合并对外部对象的引用时要特别小心

Object orientation

A programming language concept originally designed for the special purpose of simulation works more generally because much of program design can be considered “constructing a model of the real world”

最初为模拟的特殊目的而设计的编程语言概念更普遍，因为程序设计的大部分可以被认为是“构建现实世界的模型”。

Objects have both an internal structure (the variables inside them) and capabilities (the methods that can be called on them)

对象既具有内部结构(其中的变量)又具有功能(可以在其上调用的方法)。

We can think of objects as real things even people (“anthropomorphism”) which communicate with each other by making method calls

我们可以把对象看作是真实的东西，甚至是人(“拟人化”)，它们通过调用方法相互通信

Objects “decide” for themselves how a method call made on them is interpreted under the principle of “dynamic binding”

对象自己“决定”在“动态绑定”原则下对其进行的方法调用如何解释。

Standard Java programming is sequential 标准Java编程是顺序的

When the code in an object makes a method call on another object, it halts and waits for the method call to finish

当一个对象中的代码对另一个对象进行方法调用时，它会停止并等待方法调用完成

The object on which the method call is made starts executing the code, it could call a method on another object and so on

进行方法调用的对象开始执行代码，它可以调用另一个对象上的方法，依此类推

Sequence diagrams in UML illustrate this

UML中的序列图说明了这一点

Code for a method call executes in its own environment: parameter and local variables and instance variables of the object it is called on

方法调用的代码在它自己的环境中执行:被调用对象的参数和局部变量以及实例变量

When a method call terminates, code execution returns to the previous environment

当方法调用终止时，代码执行返回到前一个环境

Aliasing means separate environments can share objects

混叠意味着不同的环境可以共享对象

Concurrent programming using “threads” is a specialist topic, but one of increasing importance

使用“线程”的并发编程是一个专业主题，但其重要性也在不断增加

The most basic design principles

Classes should be written so the objects they define are things we can have an image of in our minds

在编写类时，应该让它们定义的对象是我们可以在脑海中想象的东西

The methods in objects should be defined so they represent clear operations on these things

应该定义对象中的方法，以便它们表示对这些东西的明确操作

Remember the terms “locality” and “modifiability”

记住"局部性"和"可修改性"

Code should be written so that objects can only interact through defined method calls

应该编写代码，使对象只能通过定义的方法调用进行交互

The Client–Contractor model of software 客户–承包商软件模型

One way of thinking of software structure is that an object which makes use of another object by calling a method on it and getting a return value or a side–effect is that it is like a clientmaking use of a contractor

考虑软件结构的一种方式是，一个对象通过调用另一个对象的方法并获得返回值或副作用来利用另一个对象，这就像客户机利用承包商一样

If someone has a job that needs doing, they may hire an expert who has the skills to do that job rather than do it themselves

如果有人有工作需要做，他们可能会雇佣有技能的专家来做这项工作，而不是自己去做

Modern society is able to achieve much more than a primitive society because most people have particular jobs and rely on others for other aspects of life

现代社会之所以能够取得比原始社会多得多的成就，是因为大多数人都有特定的工作，并且在生活的其他方面依赖他人

Similarly, a complex software system works because it is divided into parts, each of which has its own role, and interacts with others performing their roles

类似地，一个复杂的软件系统之所以能工作，是因为它被分成几个部分，每个部分都有自己的角色，并与执行各自角色的其他部分相互作用

If I hire someone to do a job for me, we will agree to a contract

如果我雇人为我做一件工作，我们会同意签订合同

The equivalent in a software system is the specification of a class or method

在软件系统中，等价的是类或方法的规范

Obligations and Benefits

If I hire someone to provide me with a product or service, we agree on the payment, I get the benefit of the product or service, the supplier gets the benefit of the payment

如果我雇佣某人为我提供产品或服务，我们同意付款，我获得产品或服务的收益，供应商获得付款的收益

I have the obligation to make the payment, the supplier has the obligation to provide the product or service

我有义务付款，供应商有义务提供产品或服务

For simple transactions, the benefits and obligations are laid down by state law, or by the internal regulations of the organisation we are working within

对于简单的交易，利益和义务由州法律或我们所在机构的内部规定规定

For more complex transactions, we may draw up an individual contract

对于比较复杂的交易，我们可以单独起草一份合同

When a method is called on an object in a software system, it has the obligation to return the specified result, it also has the obligation not to do anything damaging, such as change a data structure passed to it when that change was not in the specification

当在软件系统中的对象上调用方法时，它有义务返回指定的结果，它也有义务不做任何破坏性的事情，例如更改传递给它的数据结构，而这种更改不在规范中

If I write code for someone else to use, I have the obligation to make sure it meets its requirements, and perhaps the benefit of being paid for writing it

如果我编写代码供其他人使用，我就有义务确保它满足其需求，并且可能因为编写代码而获得报酬

If I am writing complex code, it may help for me to think of it as being broken down into separate pieces (classes) which describe how workers work to provide their obligations, and rely on other workers they use providing their obligations

如果我正在编写复杂的代码，它可能会帮助我把它分解成单独的部分(类)，描述工人如何工作来提供他们的义务，并依赖于他们使用的其他工人提供他们的义务

A clear establishment of the separate roles of each class, described by their specification, and an understanding of why classes should not do anything destructive outside these roles is key to good software development which is sustainable as the software grows more complex

清晰地建立每个类的独立角色，由它们的规范描述，并理解为什么类不应该在这些角色之外做任何破坏性的事情，这是良好的软件开发的关键，随着软件变得越来越复杂，这种开发是可持续的

The principles for good software design help us decide how to break down software into separate parts with clearly defined roles

好的软件设计原则帮助我们决定如何将软件分解成具有明确定义角色的独立部分

Design by Contract™

The phrase and idea of Design by Contract was developed (and trademarked) by the French Computer Scientist, Bertrand Meyer

契约式设计这个短语和理念是由法国计算机科学家伯特兰·迈耶(Bertrand Meyer)提出的(并注册了商标)

Meyer considered that client code has an obligation to meet any pre-conditions specified by the contractor code (as a simple example it may be specified that an int parameter should be set to a non-negative value)

Meyer认为客户端代码有义务满足承包商代码指定的任何先决条件(作为一个简单的例子，可以指定int参数应设置为非负值)。

From this, Meyer opposed the idea of defensive programming, which means writing your code in a way that makes sure it can handle all possible inputs

由此，Meyer反对防御性编程的想法，这意味着编写代码时要确保它能够处理所有可能的输入

Meyer proposed that methods given invalid input should terminate in a special way — which was influential in the development of Java's exception mechanism

Meyer提出，给出无效输入的方法应该以一种特殊的方式终止——这对Java异常机制的开发产生了影响

Meyer proposed that tests for whether methods have met their post-conditions be incorporated into the program itself as assertions which are now an aspect of Java

Meyer建议将方法是否满足后置条件的测试作为断言合并到程序中，这是Java的一个方面

Single Responsibility Principle (SRP) 单一职责原则

Every object in a system should have a single responsibility, and all the object's services should be focused on carrying out that single responsibility

系统中的每个对象都应该有一个单一的职责，并且所有对象的服务都应该集中于执行这个单一的职责

Leads to highly cohesive software

导致高度内聚的软件

Also expressed as:

- "A class should have only one reason to change""一个类应该只有一个改变的理由"

Cohesion and Coupling

As covered in previous teaching week:

- Cohesion: the extent to which code inside a module works with other code inside the same module 内聚性:一个模块内的代码与同一模块内的其他代码协同工作的程度

- Coupling: the extent to which code inside a module works with other code in another module 耦合:一个模块内的代码与另一个模块中的其他代码协同工作的程度

Good quality code has high cohesion and loose coupling

高质量的代码具有高内聚和松耦合

So:

- If a class has methods and variables which do not make use of each other, can it be split into separate classes? 如果一个类的方法和变量不互相使用，它可以被分割成单独的类吗?
- If code in one class makes a lot of calls of methods on objects of another class, can the classes be re-arranged to reduce the number of method calls? 如果一个类中的代码对另一个类的对象进行了大量的方法调用，是否可以重新排列这些类以减少方法调用的数量?

举个例子:

SRP Example

- Consider the following class:

```
class Employee // Example of code which breaks the SRP
{ …
 public Money calculatePay(…) { … }
 public void saveToDatabase(…) { … }
 public String reportHours(…) { … }
 …
}
```

- This means that class `Employee` has its own code to
  - Produce an object of type `Money` representing the payment to an employee
  - Save its details to a database
  - Produce a `String` which represents a recording of the employee's worked hours
- Class `Employee` will need to be changed if the pay roll calculation requirements change, if the database storage requirements change, if the format for `String`s for reporting hours worked changes.

此时我们看到，这个代码已经不符合SRP的“A class should have only one reason to change”了

- Consider the following class:

```
class Employee
{ …
 public Money calculatePay(…) { … }
 public void saveToDatabase(…) { … }
 public String reportHours(…) { … }
 …
}
```

- The code in the methods `calculatePay`, `saveToDatabase` and `reportHours` will have a lot of aspects that do not relate closely to each other, giving low cohesion
- The code in the method `calculatePay` will need to interact extensively with a `PayRoll` object, the code in the method `saveToDatabase` will need to interact extensively with a `Database` object, giving tight coupling

---

我们看此时的Employee的三个方法，互相之间没什么互动，所以是low cohension，而且和其他module（database，Payroll...）有很多的interaction，所以是tight coupling，完全不符合我们上面说的good quality code的两个点，所以我们要修改代码

- It would be better if `Employee` objects did not need to "know" about all the detail that would be required to perform the various operations
- Here is one way to resolve it (using delegation):

```
class Employee
{ …
 private PayrollCalculator1 pc;
 private DatabaseStorer1 ds;
 private HoursFormatter1 hf;
  …

 public Money calculatePay(…)
  { return pc.calculatePay(this,…) }

 public void saveToDatabase(…)
  { ds.saveToDatabase(this,…) }

 public String reportHours(…)
   { return hf.reportHours(this,…) }
 …
}
```

- With this structure, there are separate classes to perform the operations

---

此时，我们有三个独立的类来进行这三个操作

- It would be better if `Employee` objects did not need to "know" about all the detail that would be required to perform the various operations
- Here is one way to resolve it (using delegation):

```
class Employee
{ …
 private PayrollCalculator1 pc;
 private DatabaseStorer1 ds;
 private HoursFormatter1 hf;
  …
 public Money calculatePay(…)
  { return pc.calculatePay(this,…) }
 public void saveToDatabase(…)
  { ds.saveToDatabase(this,…) }
 public String reportHours(…)
   { return hf.reportHours(this,…) }
 …
}
```

- With this structure, there are separate classes to perform the operations

但是，我们看图，Database和PayRoll还直接和Employee有连接，不符合上面说的"It would be better if Employee objects did not need to 'know' about all the

detail that would be required to perform the various operations"（这里后面的dependency会解释）

更好的方法是让这些类调用Employee，把三个函数分到三个类里，很大程度提升了cohension（只有一个函数）和减少了coupling（函数数量减少了，那么和非本类连接的就少了），同样更符合SRP（一个类一个函数一个职责，只有一种原因可以改变这个类）

- A better way to resolve it is for the delegation to be the other way round:

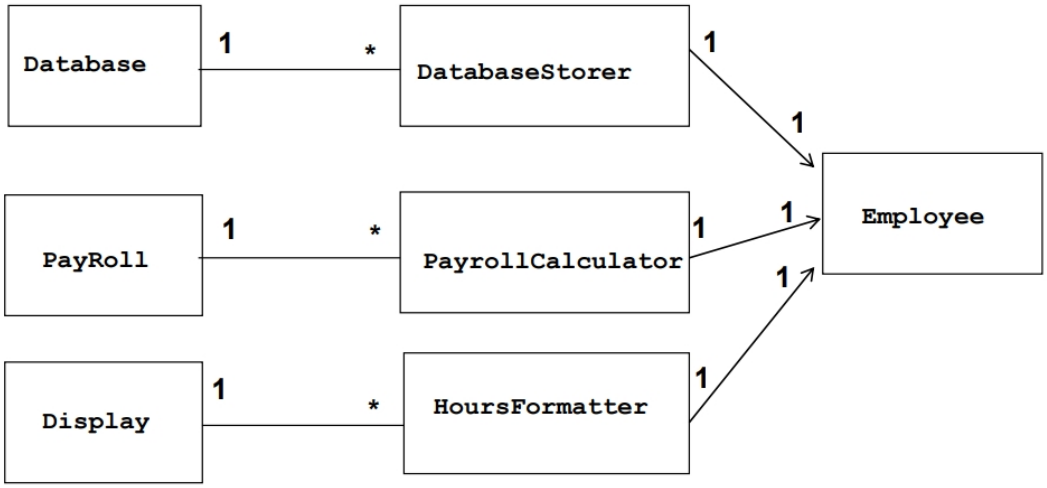
```
class PayrollCalculator
{ …
 private Employee emp;
 private PayRoll pr;
    …
 public Money calculatePay(…) { … }
}

class DataBaseStorer
{ …
 private Employee emp;
 private Database db;
    …
 public void saveToDatabase(…) { … }
}

// Similar for HoursFormatter
```

- Now the `PayRoll`, `DataBase` and `Display` objects interact with `Employee` objects indirectly through objects which have the single responsibility of managing that interaction

In this case there would be many `PayrollCalculator`, `DatabaseStorer` and `HoursFormatter` objects, one for each `Employee` object. A `PayrollCalculator`, `DatabaseStorer` or `HoursFormatter` object manages the connections between an `Employee` object and the `PayRoll`, `Database` or `Display` objects.



Dependency（结合上面的例子看）

We wrote of one class “knowing” another, but there are degrees of linkage between classes

我们写过一个类“知道”另一个类，但类之间有一定程度的联系

In our first design, an *Employee* object HAS-A *PayrollCalculator1*, a *DatabaseStorer1* and an *HoursFormatter1* object, possibly shared by other *Employee* objects. *PayrollCalculator1* etc objects do not have an *Employee* object as part of their structure, but they have a method which takes an *Employee* object and calls methods on it

在我们的第一个设计中，一个*Employee*对象HAS-A *PayrollCalculator1*，一个*DatabaseStorer1*和一个*HoursFormatter1*对象，可能被其他*Employee*对象共享。*PayrollCalculator1*等对象没有*Employee*对象作为其结构的一部分，但它们有一个方法，该方法接受*Employee*对象并在其上调用方法

In our second design, a *PayrollCalculator* object has an *Employee* object and a *PayRoll* object, the same *Employee* object will also be referenced as part of the structure of a *DataBaseStorer* object and an *HoursFormatter* object

在我们的第二个设计中，PayrollCalculator对象有一个Employee对象和一个PayRoll对象，同样的Employee对象也将作为databasestorer对象和HoursFormatter对象结构的一部分被引用

This is not an IS-A relationship (inheritance)

这不是一个is - a关系(继承)

Any linkage between classes is a “dependency”, a “weak dependency” is when a class takes object of another class as method arguments, but does not call methods on them

类之间的任何链接都是“依赖”，“弱依赖”是指一个类将另一个类的对象作为方法参数，但不调用它们的

方法

Association, Aggregation and Composition

An “association” is when an object of one class HAS-A link to an object of another class through an instance variable

“关联”是指一个类的对象通过实例变量链接到另一个类的对象

An “aggregation” is a form of association when an object of one class is made up of objects of other classes – in our second design a DatabaseStorer object is an aggregation of an Employee object and a Database object

当一个类的对象由其他类的对象组成时，“聚合”是一种关联形式——在我们的第二个设计中，databasestover对象是Employee对象和Database对象的聚合

A “composition” is a form of aggregation where the objects joined together exist only to make the aggregation – this is NOT the case with a DatabaseStorer object as it shares its Employee and Database objects with other objects which are also aggregations such as the PayrollStorer object for that Employee object

“组合”是一种聚合形式，其中连接在一起的对象只是为了进行聚合——databasestover对象不是这样，因为它与其他也是聚合的对象共享其Employee和Database对象，例如该Employee对象的payrollstover对象

Be careful – these terms are not strictly defined, different authors may use them with slightly different meanings

小心–这些术语没有严格的定义，不同的作者可能会用不同的意思来使用它们

Open-Closed Principle (OCP) 开闭原则

Software modules (classes, methods, etc) should be

软件模块(类、方法等)应该是

- “open for extension” 可扩展
- “closed for modification” 对修改封闭

Meaning：

- We can make a module behave in new and different ways as requirements change or to meet new needs 当需求发生变化或满足新的需求时，我们可以使模块以新的和不同的方式运行
- But we should be able to do this in a way that does not require changing the code of the module 但是我们应该能够以一种不需要改变模块代码的方式来做到这一点

Abstraction is the key

抽象是关键

Don’t Repeat Yourself (DRY)

Suppose you find yourself writing very similar code many times

假设您发现自己多次编写非常相似的代码

You might use “cut and paste”, copying the code and then modifying it for where it is used each time

您可以使用“剪切和粘贴”，复制代码，然后根据每次使用的位置修改它

This seems to save time, but is poor practice, for example:

这似乎可以节省时间，但这是一种糟糕的做法，

- Suppose you have code for sorting an array of integers 假设您有用于对整数数组排序的代码
- You find you need to sort an array of Strings, so you cut and paste that code, changing *n1<n2* to *str1.compareTo(str2)<0* and so on 您发现需要对字符串数组进行排序，因此剪切并粘贴该代码，将 *n1<n2*更改为*str1.compareTo(str2)<0*，以此类推
- Now suppose you find a mistake in your sort code, or you discover a more efficient sorting algorithm 现在假设您在排序代码中发现了一个错误，或者您发现了一个更有效的排序算法
- You will have to go through all the places where you used “cut and paste” to get the sort code, and change it to the new version 您必须遍历所有使用“剪切和粘贴”来获得排序代码的地方，并将其更改为新版本
- It is better to have one generalised sort method 最好有一个一般化的排序方法

Generalisation

The “Don’t Repeat Yourself” (DRY) principle is the “Open–Closed Principle” (OCP) looked at the other way round

“不要重复自己”(DRY)原则是“开闭原则”(OCP)的另一种方式

Designing and writing abstract code is hard, so we often start off writing more specific code

设计和编写抽象代码很困难，所以我们经常从编写更具体的代码开始

When we find we are repeating ourselves in code, we should see if we could replace the repeated code by writing and using more abstract code

当我们发现我们在代码中重复自己时，我们应该看看是否可以通过编写和使用更抽象的代码来替换重复的代码

- So, generalisation is identifying repeated aspects and from this moving to write more abstract code 因此，泛化就是识别重复的方面，并由此转向编写更抽象的代码

Inheritance as a way of meeting the OCP

When we declare one class as a subclass of another (its superclass), we only have to give the way it differs from the superclass

当我们将一个类声明为另一个类(它的超类)的子类时，我们只需要说明它与超类的不同之处

The subclass can add extra features (methods and instance variables that they use)

子类可以添加额外的特性(它们使用的方法和实例变量)

The subclass can change the way the methods of the superclass work (overriding)

子类可以改变超类方法的工作方式(重写)

So writing a subclass is a way of modifying the way the code of the superclass works without changing that actual code

因此，编写子类是在不改变实际代码的情况下修改超类代码工作方式的一种方式

Two ways inheritance meets the OCP

Another way in which inheritance meets the OCP is that a variable of a class type can be set to refer to an object whose type is a subclass

继承满足OCP的另一种方式是，可以将类类型的变量设置为引用类型为子类的对象

So an existing method with a parameter of a particular class will work with an object of a subclass of that class as an argument without any need to make changes to it

因此，具有特定类的参数的现有方法将与该类的子类的对象作为参数一起工作，而无需对其进行任何更改

We will consider this further when we look at interface types and the Dependency Inversion Principle, and issues with inheritance in the the Liskov Substitution Principle

当我们看到接口类型和依赖反转原则以及Liskov替代原则中的继承问题时，我们将进一步考虑这一点

The Liskov Substitution Principle suggests caution on using inheritance because of the way in which a method is overridden could result in code having an unexpected behaviour

Liskov替代原则建议在使用继承时要谨慎，因为重写方法的方式可能会导致代码出现意外行为

Parameterisation 参数化

- Longest String in an ArrayList of Strings:

```
static String longest(ArrayList<String> list)
{
 String longestSoFar="";
 for(String str : list)
    if(str.length()>longestSofar.length())
       longestSofar=str;
 return longestSoFar;
}
```

- Generalised with a Comparator parameter:

```
static String most(ArrayList<String> list, Comparator<String> comp)
{
 String mostSoFar="";
 for(String str : list)
    if(comp.compare(str,mostSoFar)>0)
       mostSofar=str;
 return mostSoFar;
}
```

就是把比较的过程放到一个比较器里，两个需要比较的作为参数

- The way the method most works can be modified without changing its code by passing different Comparator objects as its second argument 通过传递不同的Comparator对象作为第二个参数，可以在不更改代码的情况下修改方法的大部分工作方式
- compare函数
    - Here is the definition of a `Comparator<String>` that would make it return the longest `String`:

```
class LongComp implements Comparator<String>
{
 public int compare(String str1, String str2)
 {
  return str1.length()-str2.length();
 }
}
```

Types as capabilities

Class types mix implementation with application, the actual class type of an object says WHAT it can do and HOW it does it

类类型混合了实现和应用，对象的实际类类型说明了它能做什么以及如何做

An interface type says only WHAT an object can do: the methods that can be called on an object known to be of that type (or its “capabilities”)

接口类型只说明对象可以做什么:可以在已知属于该类型的对象(或其“能力”)上调用的方法。

Using interface types is not only a way of generalisation, it helps maintain the principle of separation of WHAT from HOW

使用接口类型不仅是一种泛化的方式，它还有助于维护“什么”与“如何”的分离原则

Actual and apparent types

- ```
  interface Quackable {
    public void quack();
  }
  ```

- ```
  class Duck implements Quackable
  // rest as before
  ```

- ```
  class DuckCall implements Quackable
  // rest as before
  ```

- ```
  public static void nQuacks(int n, Quackable q){
   for(int i=0; i<n; i++)
      q.quack();
  }
  ```

- `Quackable` is “the type of objects on which `quack()` can be called”

- A variable of type Quackable can refer to an object of type Duck, an object of type DuckCall, or an object of any other type which is declared as implements Quackable 一个Quackable类型的变量可以引用一个Duck类型的对象，一个DuckCall类型的对象，或者任何其他类型的被声明为实现Quackable的对象
- From this we can see an object may have more than one type, its actual type is the class of the constructor which created it 从这里我们可以看到一个对象可以有多个类型，它的实际类型是创建它的构造函数的类
- But it also has the type of any interface its actual type implements 但它也具有其实际类型实现的任何接口的类型
- The apparent type of an object referred to by a variable is the type of the variable 变量引用的对象的表面类型就是该变量的类型

Subtypes and supertypes

Quackable is a supertype of both Duck and DuckCall

Quackable是Duck和DuckCall的超型

Duck and DuckCall are both a subtype of Quackable

Duck和DuckCall都是Quackable的一个亚型

This is similar to the relationship between classes given by extends

这类似于extends给出的类之间的关系

However, a class can only extend one other class, but it can implement any number of interfaces

然而，一个类只能扩展另一个类，但它可以实现任意数量的接口

With implements, as interface types have only method headers there is no code to inherit

对于implements，由于接口类型只有方法头，所以没有代码可以继承

PPT Topic15 P71开始 Trying to generalise with complex classes 这里建议看一看，虽然也是java基础内容，但是对前面的一个小总结

The Dependency-Inversion Principle (DIP) – 依赖倒置原则

The Dependency Inversion Principle can be stated as:

依赖倒置原则可以表述为:

- High-level modules should not depend on low-level modules; instead, both should depend on abstractions 高级模块不应该依赖于低级模块;相反，两者都应该依赖于抽象
- Abstractions should not depend on details; instead, details should depend on abstractions 抽象不应该依赖于细节;相反，细节应该依赖于抽象

The attempt to generalise by creating a complex Quacker class was an example of a high-level module which is dependent on a low-level module, because the nQuacks1 method which used the Quacker class objects needed to know the details of how those objects work

通过创建一个复杂的Quacker类来进行泛化的尝试就是一个高级模块依赖于低级模块的例子，因为使用Quacker类对象的nQuacks1方法需要知道这些对象如何工作的细节

```
public static void nQuacks(int n, Quackable q){
 for(int i=0; i<n; i++)
    q.quack();
}
```

```
public static void nQuacks1(int n, Quacker q) {
 for(int i=0; i<n; i++) {
    System.out.print(q.getName()+ ": ");
    if(q.isFake())
       System.out.print(" Fake ");
    q.noise();
   }
}
```

The nQuacks method only depends on the abstraction Quackable, it leaves the details to the classes which implement that interface

nQuacks方法只依赖于抽象Quackable，它将细节留给实现该接口的类

So the nQuacks method shows an example of keeping to the DIP, and the nQuacks1 method shows an example of not keeping to the DIP

nQuacks功能显示了一个保持DIP的例子，而nQuacks1功能显示了一个不保持DIP的例子

A simple way of keeping to the DIP is always to use interface types for parameters to methods and variables in classes

保持DIP的一种简单方法是始终为方法和类中的变量的参数使用接口类型

Dependency on abstractions

The idea of abstraction is that we try to decide what are the important aspects of some type of object that we need to consider, and what are unimportant details which we can leave out of consideration

抽象的概念是，我们试图决定哪些是我们需要考虑的某一类对象的重要方面，哪些是不重要的细节，我们可以忽略不计

An abstraction is a description of a type of object just in terms of those aspects which we need to know about it

抽象是根据我们需要了解的方面对一种对象的描述

An important part of good program design is to work out what abstractions we want to define and use

好的程序设计的一个重要部分是找出我们想要定义和使用的抽象

Code which refers to objects only through variables of an interface type is dependent only on the abstraction which that interface type defines

仅通过接口类型的变量引用对象的代码仅依赖于该接口类型定义的抽象

In our example, the abstraction decided on was that we view an object just in terms of one

method which makes a particular sort of noise, we do not consider the details of the different variations of that noise

在我们的例子中，决定的抽象是，我们只根据一种产生特定噪音的方法来看待一个对象，我们不考虑该噪音的不同变化的细节

That means detailed decisions on what noise is made and how are left to the classes which implement the interface, so the details of these classes are dependent on the interfaces they implement

这意味着产生什么噪声以及如何产生噪声的详细决定留给实现接口的类，因此这些类的细节依赖于它们实现的接口

The Interface-Segregation Principle (ISP) 接口分离原则

The ISP can be expressed as “Clients should not be forced to depend on methods they do not use”

ISP可以表示为“不应该强迫客户端依赖于他们不使用的方法”。

The DIP suggests the use of interface types, but the ISP goes further and stresses the importance of small interface types

DIP建议使用接口类型，但ISP更进一步，强调小接口类型的重要性

A small interface type is one which has few methods in it

一个小的接口类型包含很少的方法

The ISP suggests if you have an interface type with a lot of methods in it (a “fat” interface), you should see if it could be broken up into several interfaces

ISP建议，如果你有一个包含很多方法的接口类型(一个“胖”接口)，你应该看看它是否可以分解成几个接口

A class can implement more than one interface, so a class that provides all the methods from the original interface can be declared as implementing all the new small interfaces

一个类可以实现多个接口，因此可以将提供原始接口中所有方法的类声明为实现所有新的小接口

However, it means that new classes which provide some of the services only have to implement those small interfaces which are relevant to them

然而，这意味着提供某些服务的新类只需要实现那些与它们相关的小接口

Classes which use the services should refer to them through variables of interface types which cover only those methods they need

使用这些服务的类应该通过接口类型的变量来引用它们，这些接口类型只包含它们需要的那些方法

就是说接口要足够小，足够精确。举个例子：

- As a simple example of the ISP, suppose instead of `Quackable` we had decided to have a general interface for duck-like objects:

```
interface DuckLike
{
 void quack();
 void fly();
}
```

- Ducks can fly, but duck-calls cannot, so if we had started off with this, we would have to give `DuckCall` a meaningless `fly` method
- Instead we can have a separate interface

```
interface Flyer
{
 void fly();
}
```

with

```
class Duck implements Quackable, Flyer
```

从Java's interface Comparable看generalisation（泛化）

- As another example of the ISP, Java provides the following interface:

```
   interface Comparable<T> {
  int compareTo(T obj)
   }
```

- Two objects of a type which implements this interface can be compared by calling `obj1.compareTo(obj2)` which returns a negative integer if `obj1` is less than `obj2`, a positive integer if `obj1` is greater than `obj2` and 0 if they are equal in "natural order"
- This enables methods to be written such as:

```
<T implements Comparable<T>> T most(ArrayList<T> list) {
T mostSoFar = list.get(0);
for(int i=1; i<array.length; i++) {
   T element=list.get(i);
   if(mostSoFar.compareTo(element)<0)
      mostSoFar=element;
    }
return mostSoFar;
}
```

（这里看不懂自行去复习一下java泛型）

the use of an interface with just the method compareTo means the code is **generalised** and does not have dependency on any aspect of the elements of the ArrayList it takes, except that they have their own method compareTo.

使用只有compareTo方法的接口意味着代码是**泛化**的，并且不依赖于它所接受的ArrayList元素的任何

方面，除了它们有自己的compareTo方法。

An additional generalisation technique here is the use of a type variable instead of a specific type of element for the ArrayList

这里的另一个泛化技术是使用类型变量代替ArrayList的特定类型元素

Generalising with Type Variables

这里！！！十分强烈建议去看PPT Topic15 P82-85，因为它上面已经讲得很明白了，而且没什么难翻译的地方，所以这里只写一点我看的时候有些疑惑的地方

<? super T>：表示?匹配的类型都是T的父类，包括T本身。

<? extends R>：表示?匹配的类型都是类型R的子类，包括R本身。

Polymorphism and Dynamic Binding 多态性与动态绑定

The ability to write code that works for many different classes is known as polymorphism (in technical terms, “poly” indicates “many”, “morph” indicates “shape” or “appearance”, “ism” indicates an “idea” or “principle”; remember “anthropomorphism”?)

编写适用于许多不同类的代码的能力被称为多态性(在技术术语中，“poly”表示“许多”，“morph”表示“形状”或“外观”，“ism”表示“想法”或“原则”;还记得“神人同形同性论”吗?)

The decision on which code to use when a method call is executed is made at run-time, it might depend on factors that are not known at compile time – this is called dynamic binding (sometimes also known as “late binding”)

执行方法调用时使用哪段代码是在运行时决定的，这可能取决于编译时不知道的因素——这称为动态绑定(有时也称为“后期绑定”)。

Mutable and immutable

```
class Rectangle {  // Is mutable
 private int height, width;

 public Rectangle(int h, int w) {
  height=h;
  width=w;
 }

 public int getHeight() {
   return height;
 }

 public void setHeight(int h) {
   height=h;
 }

 public int getWidth() {
   return height;
 }

 public void setWidth(int w) {
   width=w;
 }

 public int area() {
   return height*width;
 }
}
```

```
class Colour {  // Is immutable
 private int red, green, blue;

 public Colour(int r, int g, int b)
 {
  red=Math.max(0,Math.min(255,r));
  green=Math.max(0,Math.min(255,g));
  blue=Math.max(0,Math.min(255,b));
 }

 public int getRed() {
   return red;
 }

 public int getGreen() {
   return green;
 }

 public int getBlue() {
   return blue;
 }

 public Colour tint(double t){
   int r = Math.round(red*t);
   int g = Math.round(green*t);
   int b = Math.round(blue*t);
   return new Colour(r,g,b);
 }
```

getter和setter的区别罢了

Assignment and casting

`Rectangle rect;`

`ColouredRectangle colrect;`

（ColoutRectangle extends Rectangle）

- rect = colrect is allowed （子类可以赋给父类）
- colrect = rect is not allowed （父类不能被赋给子类）
- colrect = (ColouredRectangle) rect is used, this is called casting 投射
- If this casting was applied but rect did not refer to a ColouredRectangle object, a ClassCastException would be thrown 如果应用了这种类型转换，但rect没有引用ColouredRectangle对象，则会抛出ClassCastException

Direct and indirect subclasses

`class ColouredRectangle extends Rectangle`

`class ShadedRectangle extends ColouredRectangle`

- To distinguish, we say ShadedRectangle is a direct subclass of ColouredRectangle and an indirect subclass of Rectangle 为了区分，我们说ShadedRectangle是ColouredRectangle的直接子类和Rectangle的间接子类
- Or ColouredRectangle is the direct superclass of ShadedRectangle and Rectangle is an

indirect superclass of ShadedRectangle 或者ColouredRectangle是ShadedRectangle的直接超类，Rectangle是ShadedRectangle的间接超类

The Liskov Substitution Principle (LSP) Liskov替代原理(LSP)

Consider we have *rect* of type *Rectangle* and:

```
int h = rect.getHeight();
int w = rect.getWidth();
rect.setWidth(w+1);
int area = rect.getArea();
```

- Here, area should have the value (w+1)*h
- But suppose rect refers to an object whose actual type is Square, then area will have the value (w+1)*(h+1)
- The problem is that a subclass can override a method with one which can do anything, so long as it has the same signature 问题是，子类可以用一个可以做任何事情的方法覆盖一个方法，只要它具有相同的签名
- This can cause code to behave in an unexpected way if we did not realise a variable is actually set to refer to an object of a subclass 如果我们没有意识到变量实际上被设置为引用子类的对象，这可能会导致代码以意想不到的方式运行
- The Liskov Substitution Principle says that methods should not be overridden in a way that changes assumptions about their behaviour 利斯科夫替代原则指出，不应该以改变对其行为的假设的方式重写方法
- There are various ways of expressing the LSP such as “Subclasses must always be substitutable for the class they extend” LSP有多种表达方式，例如“子类必须总是可以替代它们所扩展的类”。
    - The assumption broken is that calling setHeight on a Rectangle changes the value returned by getHeight but not the value returned by getWidth, and calling setWidth on a Rectangle changes the value returned by getWidth but not the value returned by getHeight 打破的假设是，在矩形上调用setHeight会改变getHeight返回的值，但不会改变getWidth返回的值，而在矩形上调用setWidth会改变getWidth返回的值，但不会改变getHeight返回的值
    - We broke that by overriding setHeight and setWidth in Square to try and keep the property that in a square the height and width are always the same 我们通过在Square中重写setHeight和setWidth来打破这个问题试图保持正方形中高度和宽度总是相同的属性
- Another way of expressing it is “Inheritance should only weaken preconditions and

strengthen postconditions”另一种表达方式是“继承应该只削弱前提条件而加强后置条件”

One way of avoiding breaking the LSP is not to use inheritance – at least not in the superclass–subclass way indicated by extends in Java

避免破坏LSP的一种方法是不使用继承——至少不是用Java中extends所表示的超类–子类的方式

There is less of a problem with the interface–implementation relationship, indicated by implements in Java, because that has no code to inherit, so no problem of code in a method being unexpectedly replaced by other code in a subclass

接口–实现关系(在Java中由implements表示)的问题较少，因为它没有要继承的代码，因此不会出现方法中的代码被子类中的其他代码意外替换的问题

Composition

This is a version of *ColouredRectangle* which is not a subclass of *Rectangle*

```
class ColouredRectangle {
  private Rectangle rect;
  private Colour colour;

 public ColouredRectangle(int h, int w, Colour c) {
  rect = new Rectangle(h,w);
  colour=c;
 }

…
 public int getWidth() {
   return rect.getWidth();
 }

 public void setWidth(int w) {
   rect.setWidth(w);
 }
…
}
```

- 这个我们会在W4详细说

Overriding equals

If equals is not overridden in the class of *obj1*, then *obj1.equals(obj2)* will only return true if *obj1* and *obj2* are aliases (that is, they refer to the same object)

如果equals在*obj1*类中没有被覆盖，那么*obj1.equals(obj2)*只会在*obj1*和*obj2*是别名时返回true(也就是说，它们引用同一个对象)

It is common practice to override equals so that *obj1.equals(obj2)* will return true if *obj1* and *obj2* refer to different object with the same content

通常的做法是重写等号，这样当*obj1*和*obj2*引用具有相同内容的不同对象时，*obj1.equals(obj2)*将返回true

Assumptions for equals *equals*的假设

One general assumption for overriding equals is that obj1.equals(obj2) will return the same as obj2.equals(obj1)

重写等号的一个一般假设是obj1.equals(obj2)将返回与obj2.equals(obj1)相同的结果

Another assumption is that if obj1.equals(obj2) returns true and obj2.equals(obj3) returns true then obj1.equals(obj3) should return true

另一个假设是，如果obj1.equals(obj2)返回true, obj2.equals(obj3)返回true，那么obj1.equals(obj3)应该返回true

However, suppose obj2 refers to an object of a subclass of obj1 then dynamic binding means that obj2.equals(obj1) would not use the same code as obj1.equals(obj2) if equals was overridden in the class of the object that obj2 refers to

然而，假设obj2引用obj1子类的对象，那么动态绑定意味着如果在obj2引用的对象的类中覆盖了equals，那么obj2.equals(obj1)将不会使用与obj1.equals(obj2)相同的代码

- This would be an issue in overriding the method equals in the classes Rectangle and ColouredRectangle given previously 这在重写前面给出的Rectangle和ColouredRectangle类中的equals方法时会出现问题

Issues with overriding equals

Code for overriding `equals` in `Rectangle`:

```
class Rectangle {
 private int height,width;
 …
 public boolean equals(Object obj)  {
  if(obj instanceof Rectangle ) {
    Rectangle rect = (Rectangle) obj;
    return height==rect.height&&width=rect.width;
  }
  else return false;
}
```

The parameter type is Object rather than Rectangle in order to properly override the equals method inherited from Object

参数类型是Object而不是Rectangle，以便正确地覆盖从Object继承的equals方法

Another issue is that rect1.equals(rect2) would change its value after a call of setHeight or setWidth

另一个问题是rect1.equals(rect2)会在调用setHeight或setWidth后改变其值

It is up to the designer to decide whether equals should be a permanent or temporary property with mutable objects

由设计者决定equals应该是可变对象的永久属性还是临时属性

Summary and self test（复习省流版）

- Java基础知识（考试也有可能让你写一点点代码）
- Aliasing and mutability 别名和可变性
  - More than one variable may refer to an object, this is called aliasing
    - aliasing的好处坏处?
  - Did you understand what is meant by “mutable” and “immutable” and why aliasing makes these an aspect that needs to be considered in design? 你理解“可变”和“不可变”是什么意思吗?为什么别名会使这些成为设计中需要考虑的一个方面?
    - 两者区别在于有无setter（上面有例子）
  - Did you understand what is meant by “overriding equals” and how this fits in with aliasing and mutability? 你是否理解“重写等号”是什么意思，以及它是如何与混叠和可变性相适应的?
- Object Oriented Programming 面向对象编程
  - Object oriented programming languages introduce new complexities into coding which have been found useful because they fit closely into design concepts 面向对象编程语言在编码中引入了新的复杂性，这些复杂性非常有用，因为它们非常适合设计概念
  - Did you understand the idea of program design as about building a model of communicating entities, where communication is actually done by method calls, and entities are represented by code objects? 您是否理解程序设计的思想是建立一个通信实体的模型，其中通信实际上是通过方法调用完成的，而实体是由代码对象表示的?
  - Did you understand how the structure of object oriented programming helps you break a program into parts which can be considered independently? 你是否理解面向对象编程的结构如何帮助你将程序分解成可以独立考虑的部分?
  - This idea is captured in the terms “locality” and “modifiability”
    - Locality – the implementation of a module can be read or written without having to examine the implementation of any other module 局部性——一个模块

的实现可以被读取或写入，而不必检查任何其他模块的实现

        - Modifiability – a module can be reimplemented without requiring changes to any module which makes use of it 可修改性——一个模块可以被重新实现，而不需要改变使用它的任何模块
        - 【参考Decomposition标题下内容】

- Type Hierarchy 类型分级结构
    - Did you understand the idea of abstraction and generalisation as key aspects of program design? 你理解抽象和泛化作为程序设计的关键方面吗?
        - Abstraction is identifying the essential aspects of some situation in order to model it in software 抽象是识别某些情况的基本方面，以便在软件中对其建模
        - Generalisation is identifying common elements in software requirements so we can provide them in reusable code 泛化是识别软件需求中的公共元素，以便我们可以在可重用的代码中提供它们
        - 【参考标题Abstruction and Generalisation下内容】
    - Did you understand how Java's type hierarchy helps support abstraction and generalisation?
        - 您了解Java的类型层次结构如何帮助支持抽象和泛化吗?
            - 【从Java's interface Comparable看generalisation 标题下内容可供参考，以及DIP的内容也可作为参考】
    - Did you understand the use of interface types in Java, in particular the way they lead to types being seen in terms of capabilities rather than implementation? 您是否理解Java中接口类型的使用，特别是它们导致类型被视为功能而不是实现的方式?
        - Much of this is summarised in the idea that you should "code to the interface" 这其中的大部分可以总结为你应该"为接口编写代码"的想法。
    - Do you understand the idea of "dynamic binding" in object-oriented programming languages, and how this leads to the Liskov Substitution Principle? 您是否理解面向对象编程语言中的"动态绑定"概念，以及它是如何导致Liskov替代原则的?
        - The decision on which code to use when a method call is executed is made at run-time, it might depend on factors that are not known at compile time – this is called dynamic binding (sometimes also known as "late binding") 执行方法调用时使用哪段代码是在运行时决定的，这可能取决于编译时不知道的因素——这称为动态绑定(有时也称为"后期绑定")。

- Six Design Principles
    - Single Responsibility Principle (SRP)

- Open–Closed Principle (OCP)
- Do not Repeat Yourself principle (DRY)
- Liskov Substitution Principle (LSP)
- Interface–Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)
    - The “Do not Repeat Yourself” principle is really just another way of expressing the “Open Closed” principle “不要重复自己”原则实际上只是“开闭”原则的另一种表达方式
    - This leaves five principles, and from the initials of each, they are often called **the SOLID design principles** 这就留下了五个原则，根据每个原则的首字母缩写，它们通常被称为SOLID设计原则

Do you understand each of these to the point where you could write a brief explanation if asked?

如果有人问你，你对这些问题的理解是否达到了可以写一个简短解释的程度?

Do you understand each of these to the point where you could write your own example of the principle being kept to or the principle being broken, if asked?

如果有人问你，你对这些原则的理解是否达到了可以写出自己遵守或违反原则的例子的程度?

【个人认为是暗示了考题里一定会有对六个设计原则其中一个或多个的解释以及举例，具体还需要等复习的时候求证，但都背都会绝对没错】